

COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY
COCHIN UNIVERSITY COLLEGE OF ENGINEERING
KUTTANADU



DECEMBER 2025

PROJECT REPORT ON

PROJECTHUB – PROJECT MANAGEMENT SYSTEM

Submitted on partial fulfillment of the requirement for the award of the degree in Master of
Computer Applications from COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY

Submitted by

ALBERT JOSEPH (24100814)

JOYAL JAMES (24030437)

NOEL MARTIN (24100394)

YADUKRISHNA.S(24030421)

DIVISION OF COMPUTER APPLICATIONS

2024-2026

COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY
COCHIN UNIVERSITY COLLEGE OF ENGINEERING
KUTTANADU



CERTIFICATE

This is to certify that this project report entitled “**PROJECTHUB – PROJECT MANAGEMENT SYSTEM**” is a bonafide record on partial fulfillment for the Degree of the Master of Computer Applications to the COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY through DIVISION OF COMPUTER APPLICATIONS, COCHIN UNIVERSITY COLLEGE OF ENGINEERING KUTTANADU, ALAPPUZHA done by **ALBERT JOSEPH** (24100814), **JOYAL JAMES** (24030437), **NOEL MARTIN** (24100394), **YADUKRISHNA S**(24030421) in the year 2025.

Project Guide

Ms. DIVYA G

Asst. Professor

CUCEK

Project Coordinator

Ms. DIVYA G

Asst. Professor

CUCEK

Head of the Department

Dr. PREETHA MATHEW K

Principal

CUCEK

DECLARATION

We hereby declare that the project entitled “**PROJECTHUB – PROJECT MANAGEMENT SYSTEM**” submitted to the DIVISION OF COMPUTER APPLICATIONS, COCHIN UNIVERSITY COLLEGE OF ENGINEERING, KUTTANADU in the partial fulfilment of the requirements for the award of Degree in MASTER OF COMPUTER APPLICATIONS is a record of original work done by us under the guidance of **Ms. DIVYA G**, Assistant Professor in MCA Department during my period of studying at Cochin University College of Engineering, Kuttanadu.

Place:

ALBERT JOSEPH

Date:

JOYAL JAMES

NOEL MARTIN

YADUKRISHNA S

ACKNOWLEDGEMENT

We are thankful to God almighty for the blessings in the successful completion of our mini project “**PROJECTHUB – PROJECT MANAGEMENT SYSTEM**”. We would like to record my profound gratitude to **Dr. PREETHA MATHEW K**, Principal and Head of the Department, MCA, Cochin University College of Engineering who has deeply inspired us to do our project. It's grateful to express our thanks to **Ms. DIVYA G**, Assistant Professor in MCA Department our Project guide, Cochin University College of Engineering, Kuttanad, because of her effective guidance, constructive criticism and innovative and useful stream of suggestions that helped us to complete our project. We are thankful to various resources that provide requirements for our projects, because requirements are backbone of every project. We are also thankful to our teachers, friends, family members, for their support and prayer for us to complete our project.

ABSTRACT

ProjectHub is designed to be an advanced project management system aimed at providing project repository services within an educational institution, and simplifying student-project guide interactions by providing an interface for the guides to monitor a collaboration of allocated students, throughout the design and development phase. Presently, there is no provision that facilitates this process other than traditional, manual reviews and periodic meetings. ProjectHub aims to streamline this process through an in-house repository system for the campus, where students can create their projects, make timely submissions to their allocated guides, and in time, store their finished projects securely among those of other students, from prior batches as well. ProjectHub also features an AI-powered Assistant “HubMaster” to guide both students and teachers throughout their activities, and provide assistance to the students right from abstract creation to documentation and deployment.

The development of this system incorporates a structured design approach, focusing on reliability, usability, and security. The platform enables real-time updates, an intuitive UI, and accurate data handling, which contribute to improved operational accuracy. Technologies used in the system ensure fast processing, data integrity and seamless interaction across various modules.

Overall, ProjectHub provides an effective solution for overcoming the challenges faced in traditional project development methods. With its scalable architecture, the system is well-positioned for future enhancements such as mobile app integration, up scaling to include more departments, further increasing its value and applicability in modern educational institutions.

CONTENTS

1. INTRODUCTION.....	01
1.1 ABOUT THE PROJECT	
1.2 OBJECTIVE AND SCOPE OF THE PROJECT	
1.3 DEFINITION OF PROBLEM	
2. SYSTEM ANALYSIS.....	07
2.1 EXISTING SYSTEM	
2.1.1 DISADVANTAGES	
2.2 PROPOSED SYSTEM	
2.2.1 ADVANTAGES	
2.2.2 MODULES	
2.3 FEASIBILITY STUDY	
3. SYSTEM REQUIREMENTS AND SPECIFICATIONS.....	15
3.1 HARDWARE CONFIGURATIONS	
3.2 SOFTWARE CONFIGURATIONS	
3.3 TECHNOLOGY USED	
3.4 PLATFORM USED	
4. SYSTEM DESIGN.....	20
4.1 DATABASE DESIGN	
4.2 DATAFLOW DIAGRAM	
4.2.1 DFD LEVEL 0	
4.2.2 DFD LEVEL 1.0 ADMIN	
4.2.3 DFD LEVEL 1.1 STUDENT	
4.2.4 DFD LEVEL 1.2 TEACHER	
4.3 CLASS DIAGRAM	
4.4 SEQUENCE DIAGRAM	

4.5 ER DIAGRAM	
4.6 ACTIVITY DIAGRAM	
4.7USE CASE DIAGRAM	
5. SYSTEM IMPLEMENTATION, TESTING AND MAINTENANCE.....	32
6.SAMPLE CODE	39
7.SCREENSHOTS	56
8. CONCLUSION	68
9.REFERENCES.....	70

LIST OF FIGURES

4.2.1 DFD LEVEL 0.....	24
4.2.2 DFD LEVEL 1.1 ADMIN.....	24
4.2.3 DFD LEVEL 1.2 STUDENT.....	25
4.2.4 DFD LEVEL 1.3 TEACHER.....	26
4.3 CLASS DIAGRAM.....	27
4.4 SEQUENCE DIAGRAM	28
4.5 ER DIAGRAM.....	29
4.6 ACTIVITY DIAGRAM.....	30
4.7USE CASEDIAGRAM.....	31

INTRODUCTION

1.1 ABOUT THE PROJECT

Project Hub is a centralized web-based platform engineered to manage the entire lifecycle of academic projects. It bridges the gap between three key stakeholders—Students, Teachers, and Administrators—by providing a unified environment for project creation, submission, mentorship, and evaluation.

The system is architected on a modern, decoupled stack. The client-side interface is built with HTML and JavaScript, communicating via HTTP requests to a high-performance FastAPI (Python) backend. Data persistence is handled efficiently using MongoDB, supplemented by local file storage for document management.

A defining feature of Project Hub is its intelligent "Chatbot Interface," powered by a Retrieval-Augmented Generation (RAG) architecture. By integrating Large Language Models (such as Gemini or Ollama) with a ChromaDB vector store, the system provides AI-driven guidance and context-aware data processing to assist users throughout their project journey.

1.2 OBJECTIVE & SCOPE OF THE PROJECT

OBJECTIVE

The primary objective of this project is to design, develop, and deploy a centralized "Project Hub". This web-based platform serves as an integrated ecosystem for managing the complete lifecycle of academic projects.

The key objectives derived from the system diagrams are:

- To Centralize Project Management: Create a single, unified platform where Students can create projects, form teams, and submit their work.
- To Streamline Guidance and Evaluation: Facilitate the interaction between Students and Teachers by allowing teachers to be notified of projects , request guideship , track student progress , and provide formal feedback and approval.
- To Define and Manage User Roles: Clearly delineate and manage the functionalities for three distinct user roles: Student (project creation and execution) ,Teacher (guidance and evaluation) , and Admin (system oversight and user management).
- To Integrate AI-Powered Assistance: Enhance the user experience by incorporating an AI and Data Processing component. This includes a Chatbot , built on a RAG Architecture with models like Gemini/Olama , to assist both Students and Teachers.
- To Ensure Persistent and Secure Data: Maintain a robust database system using MongoDB and Local File Storage to securely store user data, project details, submissions, and communications.

SCOPE

The scope of this project defines its functional and technical boundaries. The system is designed to handle all core activities related to academic project management, from inception to final approval.

Functional Scope by User Role:

1 Student:

1.3 Authentication: Sign in / Sign up to the platform.

1.4 Project Management: Create new projects, add/view project details, and choose team members.

1.5 Team: Send and manage Team_invitations.

1.6 Guidance: Receive and accept guideship requests from teachers.

1.7 Submission: Submit project work by sending Project_links.

1.8 Interaction: Communicate with teachers via a chat/comment system and receive feedback.

1.9 Resources: Use the AI chatbot and browse a database of senior projects.

2 Teacher:

2.3 Authentication: Sign in / Sign up to the platform.

2.4 Guidance: View unassigned projects , request guideship , and manage their assigned projects.

2.5 Evaluation: View student submissions on a dashboard , track project progress , and formally Accept/reject submissions.

2.6 Interaction: Provide feedback and comments on submissions using the project_chat_messages system.

2.7 Resources: Use the AI chatbot (which connects to a chromaDb) to assist in evaluation or guidance.

3 Admin:

3.3 Authentication: A dedicated Admin login.

3.4 User Management: "Overall system control" including the ability to Add / Remove user and manage users.

3.5 Project Oversight: manage projects across the entire system.

3.6 Analytics: Access system statistics (loadAdminStats).

Technical Scope:

- Frontend: A web browser application built with HTML and JS.
- Backend: A FASTAPI (Python) server that handles all business logic and API requests.
- Database: A MongoDB database serves as the primary data store for users, projects, links, and messages.
- AI Subsystem: A RAG Architecture that uses a VECTOR DB(identified as chromaDB) and Gemini/Ollama models to power the chatbot service.
- Storage: Local File Storage is used for handling file-related services

3.7 DEFINITION OF PROBLEM:

Key Problems Addressed:

1. **Fragmented Workflow:** Replaces scattered tools (e.g., email, file sharing) with a unified platform for formal team formation, guide assignment, and contextual feedback.
2. **Lack of Progress Tracking:** Eliminates visibility gaps by providing Teachers and Admins with real-time tracking of project status, milestones, and submissions.
3. **Inefficient Knowledge Transfer:** Solves information bottlenecks by allowing students to browse senior projects and receive instant answers via an AI-powered RAG chatbot.
4. **Administrative Overhead:** Removes manual burdens by providing a centralized Admin dashboard for managing users and project oversight.

SYSTEM ANALYSIS

2.1 EXISTING SYSTEM

The current system appears to be a fragmented, manual, and decentralized set of processes that lacks the integration and intelligence of the proposed platform.

The primary limitations of the existing system are:

- **No Centralized Platform:** There is no single, unified web-based hub. Students, teachers, and administrators must rely on a collection of disconnected tools, such as email for communication and feedback, separate file-sharing services for submissions, and manual spreadsheets or paper forms for tracking.
- **Manual and Ad-Hoc Workflows:**
 - **Guide Assignment:** The process for a teacher to become a guide for a project is not formalized. It likely relies on manual email requests or in-person discussions, leading to inefficiency and a lack of transparency.
 - **Team Formation:** There is no formal system for students to create teams and manage memberships, making it difficult to track who is working with whom.
 - **Submission Process:** Submissions are not centrally logged. Teachers may receive project links or files via email, making it difficult to track submission times, versions, or phases.
- **Lack of Real-Time Tracking:** It is extremely difficult for administrators and teachers to get a real-time overview of project status. Tracking progress is a manual data-collection task, rather than a feature of an integrated dashboard.

- Inaccessible Knowledge and Support:
 - Past projects ("senior's projects") are not stored in an accessible "project database," preventing new students from learning from previous work.
 - Support for students and teachers is manual and not on-demand. There is no AI-powered "chatbot" to provide instant answers, placing a higher support burden on faculty and peers.
- High Administrative Overhead: System administrators lack a central "Admin login" or dashboard for "Overall system control". All user management (adding/removing users) and project oversight is likely a manual, time-consuming process.

2.2 PROPOSED SYSTEM

The proposed system is a comprehensive "Project Hub," a web-based platform designed to centralize and automate the management of academic projects. It replaces fragmented manual processes with a unified ecosystem that connects Students, Teachers, and Admins, streamlining the entire lifecycle from project inception to final evaluation.

The system is defined by its modern architectural components and core workflows:

- Centralized Web Architecture: The platform is accessible via a standard Web Browser using a frontend built with HTML and JavaScript. This frontend communicates with a high-performance BackendServer powered by FASTAPI (Python), which orchestrates all user requests and services.
- Integrated Data Management: The system utilizes MongoDB as its primary database to store structured data for users, projects, and messages. It allows for a unified user

management system where Students, Teachers, and Admins securely log in to access role-specific dashboards. Additionally, Local File Storage is integrated for handling project document submissions.

- **Intelligent AI Assistant (RAG Architecture):** A distinguishing feature is the embedded AI and Data Processing module. The system employs a RAG (Retrieval-Augmented Generation) Architecture that utilizes Gemini or Olama models. It connects to a Vector Database (ChromaDB) to retrieve relevant information, allowing users to interact with a Chatbot Interface for guidance, abstract creation, or querying the project database.

2.2.1 ADVANTAGES

- **Centralized& Unified Platform:** The system consolidates all project activities—creation, team formation, submission, and grading—into a single interface, eliminating the need for scattered emails or separate file-sharing tools.
- **AI-Powered Knowledge Support:** The integration of an AI **Chatbot** and **Vector Database** allows users to instantly access information. Students can "**Browse senior's projects**" to learn from past work, and the AI can assist in assessing new projects, significantly enhancing the learning and guidance process.
- **Real-Time Progress Tracking:** The system provides transparency through status updates. Teachers can actively "**Track student progress**" and view the "Status" and "Progress" attributes of any project in real-time, ensuring no student falls behind.

- **Structured Communication & Feedback:** The dedicated "**project_chat_messages**" feature ensures that all feedback is contextual and recorded. Teachers can "**Provide feedback in chats**" directly related to specific phases, making communication clear and retrievable.

- **Efficient Workflow Automation:** Automated notifications for key events—such as when a student is "**Notified about guideship request**" or a teacher receives a "**notification of submission**"—reduce delays and ensure timely responses from all parties.

- **Role-Based Security:** The system enforces strict role boundaries (Student, Teacher, Admin), ensuring that users only access features relevant to their duties, such as "**Admin login**" for system control or "**Teacher**" dashboards for evaluation.

2.2.2 MODULES

- User Management & Authentication Module
- Project & Team Management Module
- Guidance & Submission Module
- Feedback & Communication Module
- AI Chatbot Module
- Knowledge Base Module
- Admin Control Panel

2.3 FEASIBILITY STUDY

The feasibility study analyzes the viability of the proposed **Project Hub** across technical, operational, and economic dimensions to ensure the project can be successfully implemented using the defined architecture.

1. Technical Feasibility: -

- **Technology Stack:** The proposed system is technically feasible. The architecture diagram (Page 1) specifies a modern, well-defined technology stack:
 - **Backend:**FASTAPI (Python), which is a high-performance framework.
 - **Frontend:** Standard Web Browser application using HTML/JS.
 - **Database:**MongoDB, a scalable and flexible NoSQL database well-suited for the nested data structures shown in the Class Diagram (e.g., users, Project).
- **AI Integration:** The most complex component is the "AI and Data Processing" module. This involves a RAG Architecture, a VECTOR DB(also called chromaDb), and models like Gemini/Olama. While this is an advanced feature, it is a well-established architecture, making it feasible provided the development team possesses the necessary expertise in AI and vector databases.
- **Conclusion:** The project is technically feasible. The components are clearly identified and compatible. The primary challenge will be the successful implementation of the RAG architecture.

2. Economic Feasibility: -

The system is economically feasible as it minimizes development and maintenance costs by relying on open-source tools.

- **Development Costs:** The system relies primarily on open-source technologies (Python, MongoDB, FastAPI, Olama), significantly reducing software licensing costs.
- **Infrastructure:** The application is lightweight enough to run on standard institutional servers or cloud tiers. The integration of Local File Storage avoids the need for expensive proprietary cloud storage solutions for student files.
- **AI Cost Management:** By supporting Olama, the system offers an option to run LLMs locally without incurring per-token API costs associated with some cloud AI providers, making it sustainable for an academic budget.
- **Conclusion:** The project is economically feasible, utilizing cost-effective software solutions that minimize ongoing operational expenses.

3. Operational Feasibility: -

This assesses how well the proposed system solves the problems of and fits into the existing operational environment.

- **User Acceptance:** The system is highly feasible from an operational standpoint. It is designed around three familiar roles: Student, Teacher, and Admin. The functions provided directly address the day-to-day operations of an academic institution.
- **Workflow Automation:** The system automates and streamlines core academic processes.
For example:

- It formalizes the guide assignment process (a Teacher Requests guideship , and a Student Accepts guideship).
 - It centralizes submissions (Finalises each project submissions).
 - It creates a dedicated channel for feedback (Provide feedback in chats).
- Conclusion: The system has high operational feasibility. It directly maps to the existing workflows of its intended users and solves the identified problems of a fragmented, manual process.

4. Time Feasibility: -

The project is highly feasible to complete within the scheduled timeframe due to three key factors:

- Modular Architecture: The Data Flow Diagrams show clear separation between modules like User Management, Project Handling, and AI Services. This allows different team members to develop features in parallel, significantly reducing total development time.
- Rapid Development Stack: The use of FastAPI (Python) and MongoDB accelerates the coding process by providing built-in tools for quick API creation and flexible data storage, avoiding the delays of complex SQL migrations.
- Phased Integration: The system design supports a phased approach: core features (Sign In, Project Creation) can be finished first, while the advanced RAG Architecture and Chatbot can be integrated as a second phase, ensuring the main project deadline is met.

SYSTEM REQUIREMENTS
AND
SPECIFICATIONS

3.1 HARDWARE CONFIGURATIONS

The selection of hardware is very important in the existence and proper working of any of the software. When selecting hardware, the size and capacity requirements are also important. This software is able to run with following hardware configuration.

Processor	: Intel core i3
Processor Speed	: 1.5GHz to 2.2GHz
RAM	: 4GB or above
Hard Disk	: 3GB and above
Keyboard	: 108 keys
Clock speed	: 500 MHz
System bus	: 32 bits

3.2 SOFTWARE CONFIGURATIONS

One of the most difficult tasks is selecting software, once the system requirement is found out, then we have to determine whether a particular software package fits for those system requirements. This section summarizes the application requirement.

Operating System: Windows 7/8/8.1/10/11.

Language: Python, Html ,Css

IDE: VS Code

Database: MongoDB

3.3 TECHNOLOGY USED

The Project Hub utilizes a modern, full-stack architecture combining web standards with advanced AI integration. The specific technologies identified in the system architecture and data flow diagrams are:

1. Frontend Technologies

- **HTML & JavaScript:** The user interface is built using standard web technologies, specifically HTML and JS (JavaScript). This ensures the application is lightweight and compatible with any standard Web Browser.

2. Backend Technologies

- **Python:** The core server-side logic is developed in Python, a versatile language chosen for its strong support for web frameworks and AI integration.
- **FastAPI:** The backend server is powered by FASTAPI, a high-performance web framework for building APIs with Python. It handles HTTP Requests from the client and manages communication between the database and AI services.

3. Database Technologies

- **MongoDB:** The primary database for the application is MongoDB. It is used to store structured data such as users, projects, submissions, and messages, as detailed in the Class Diagram.
- **ChromaDB (Vector DB):** The system utilizes a VECTOR DB, specifically identified as chroma DB in the Data Flow Diagram. This is essential for the AI component, storing vector embeddings to enable efficient retrieval for the RAG architecture.
- **Local File Storage:** The system also employs Local File Storage for managing physical files (likely project documents or reports) uploaded by students.

4. AI and Machine Learning Technologies

- **RAG Architecture:** The system implements a RAG (Retrieval-Augmented Generation) Architecture. This framework combines information retrieval with text generation to provide accurate, context-aware responses.
- **Gemini / Olama:** The AI processing is driven by Large Language Models (LLMs). The architecture explicitly lists Gemini and Olama as the providers for these models, allowing the Chatbot Interface to assist users.

3.4 PLATFORM USED

The Project Hub is designed as a web-based platform, ensuring cross-platform compatibility and ease of access.

- **Web Platform (Client-Side):** The application is platform-independent on the client side. It is accessed through a standard Web Browser, making it available on any operating system (Windows, macOS, Linux, Android, iOS) that supports modern web browsing. The frontend interfaces are built using HTML and JS, which run natively in the browser environment.
- **Server-Side Platform:** The backend services are hosted on a server environment capable of running Python applications.
 - **Application Server:** The core logic runs on FASTAPI, a high-performance web framework.
 - **Data Platform:** The system relies on MongoDB for document-oriented data storage.
 - **AI Platform:** The system integrates with Gemini and Olamato provide AI capabilities, utilizing a RAG Architecture backend by a VECTOR DB (ChromaDB).
- **Storage Platform:** The system utilizes Local File Storage for managing physical files uploaded by users, integrated directly with the backend server.

SYSTEM DESIGN

4.1 DATABASE DESIGN

The system utilizes a Hybrid Database Architecture, combining a NoSQL document store (MongoDB) for structured application data and a Vector Database (ChromaDB) for high-dimensional AI data.

Database Schema

USER

Column Name	Datatype	Nullable	Default
User_id	Objectid	No	generatedBy Default
fullName	String	No	NULL
email	String	No	NULL
hashedPassword	String	No	NULL
role	String	No	NULL
registrationNumber	Integer	No	NULL
department	String	No	NULL
stickyNotes	String	No	NULL
securityQuestion	String	No	NULL
securityAnswerHash	String	No	NULL
createdAt	Datetime	No	NULL

PROJECT

Column Name	Datatype	Nullable	Default
Project_id	Object_id	No	generatedbyDefault
Course_code	String	No	NULL
Name	String	No	NULL
Deadline	String	Yes	NULL
Description	String	Yes	NULL
Owner_id	String	No	NULL
Department	String	No	NULL
Status	String	No	'Unassigned'
Progress	String	No	'0%'
Guide_id	String	Yes	NULL
Guide_name	String	Yes	NULL
Team_members	String[]	No	NULL
Milestones	String[]	No	NULL

GUIDE_REQUESTS

Column Name	Datatype	Nullable	Default
Guide_request_id	ObjectId	No	generatedByDefault
Project_id	ObjectId	No	NULL
Project_name	String	No	NULL
Teacher_id	ObjectId	No	NULL
Teacher_name	String	No	NULL
Owner_id	ObjectId	No	NULL
Owner_name	String	No	NULL
Status	String	No	'Pending'
declineReason	String	No	NULL
createdAt	Datetime	No	CurrentTimestamp
respondedAt	Datetime	Yes	NULL

TEAM INVITATION

Column Name	Datatype	Nullable	Default
Project_id	ObjectId	NO	NULL
Project_name	String	NO	NULL
Inviter_id	ObjectId	NO	NULL
Inviter_name	String	NO	NULL
Invitee_id	ObjectId	NO	NULL
Invitee_name	String	NO	NULL
Status	String	NO	'Pending'

PROJECT_LINKS

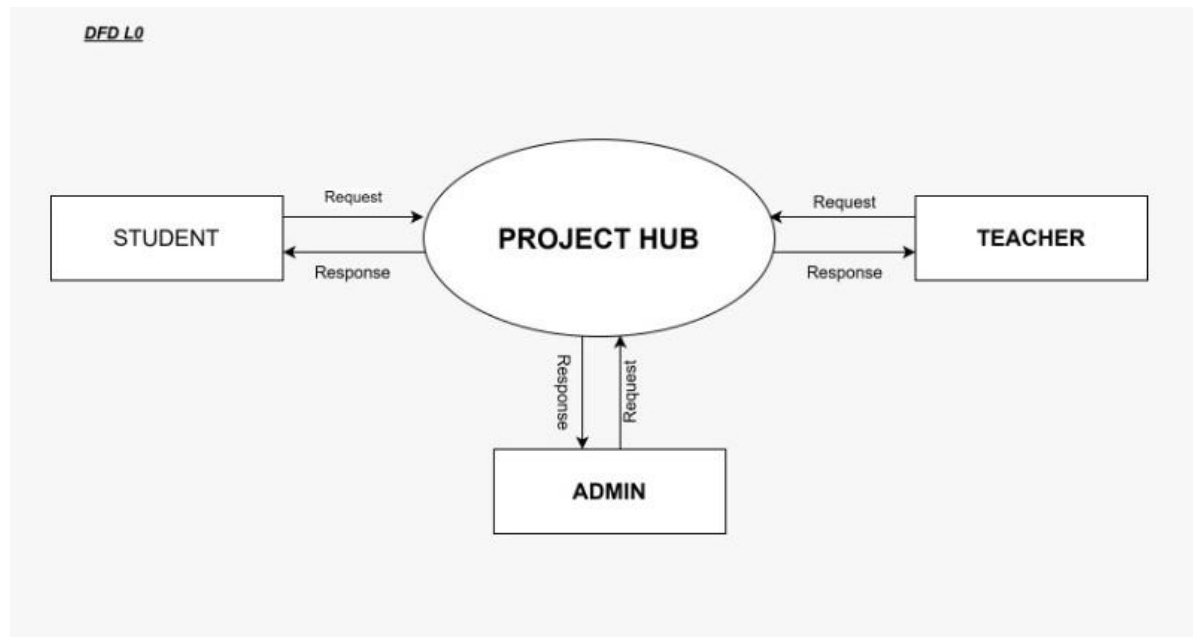
Column Name	Datatype	Nullable	Default
Project_id	ObjectId	No	NULL
Phase_order	Integer	No	NULL
linkURL	String	No	NULL
linkDescription	String	Yes	NULL
submittedByUserId	ObjectId	No	NULL
submittedByUserName	String	No	NULL
submittedAt	Datetime	No	CurrentTimestamp

PROJECT_CHAT_MESSAGES

Column Name	Datatype	Nullable	Default
Project_id	ObjectId	NO	NULL
PhaseOrder	Integer	NO	NULL
senderId	ObjectId	NO	NULL
senderName	String	NO	NULL
senderRole	String	NO	NULL
messageText	String	NO	NULL
sentAt	Datetime	NO	NULL

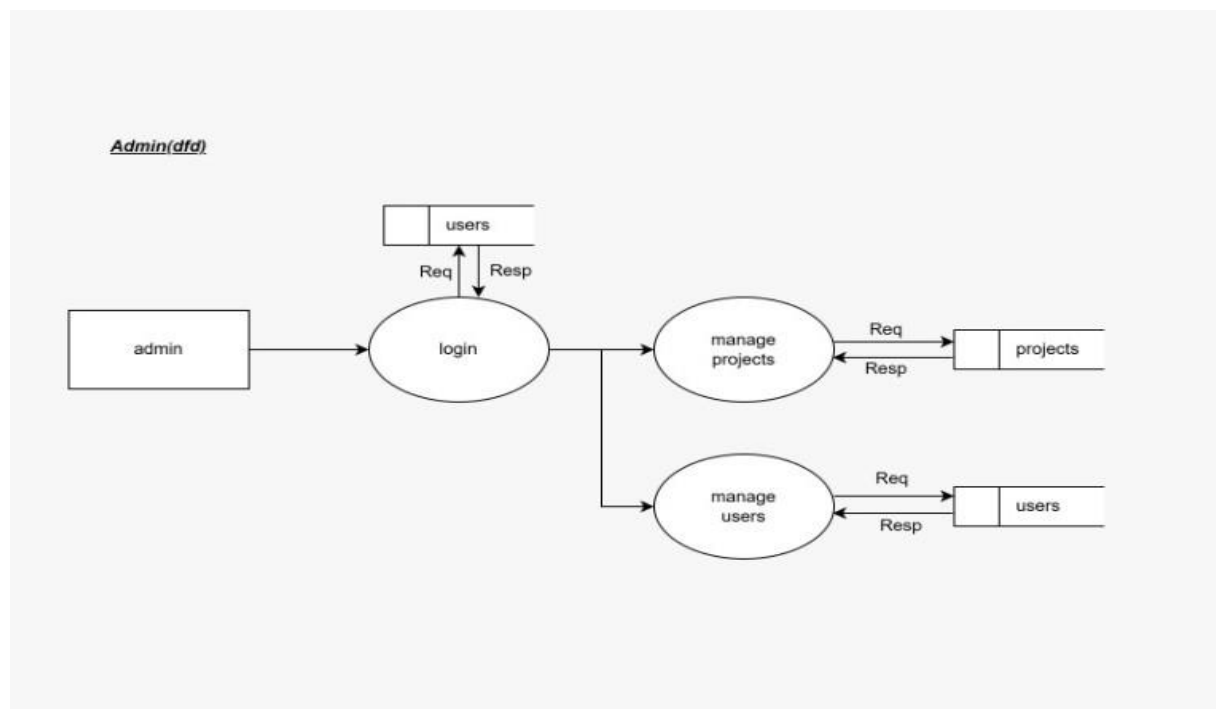
4.2 DATA FLOW DIAGRAM

4.2.1 DFD LEVEL 0



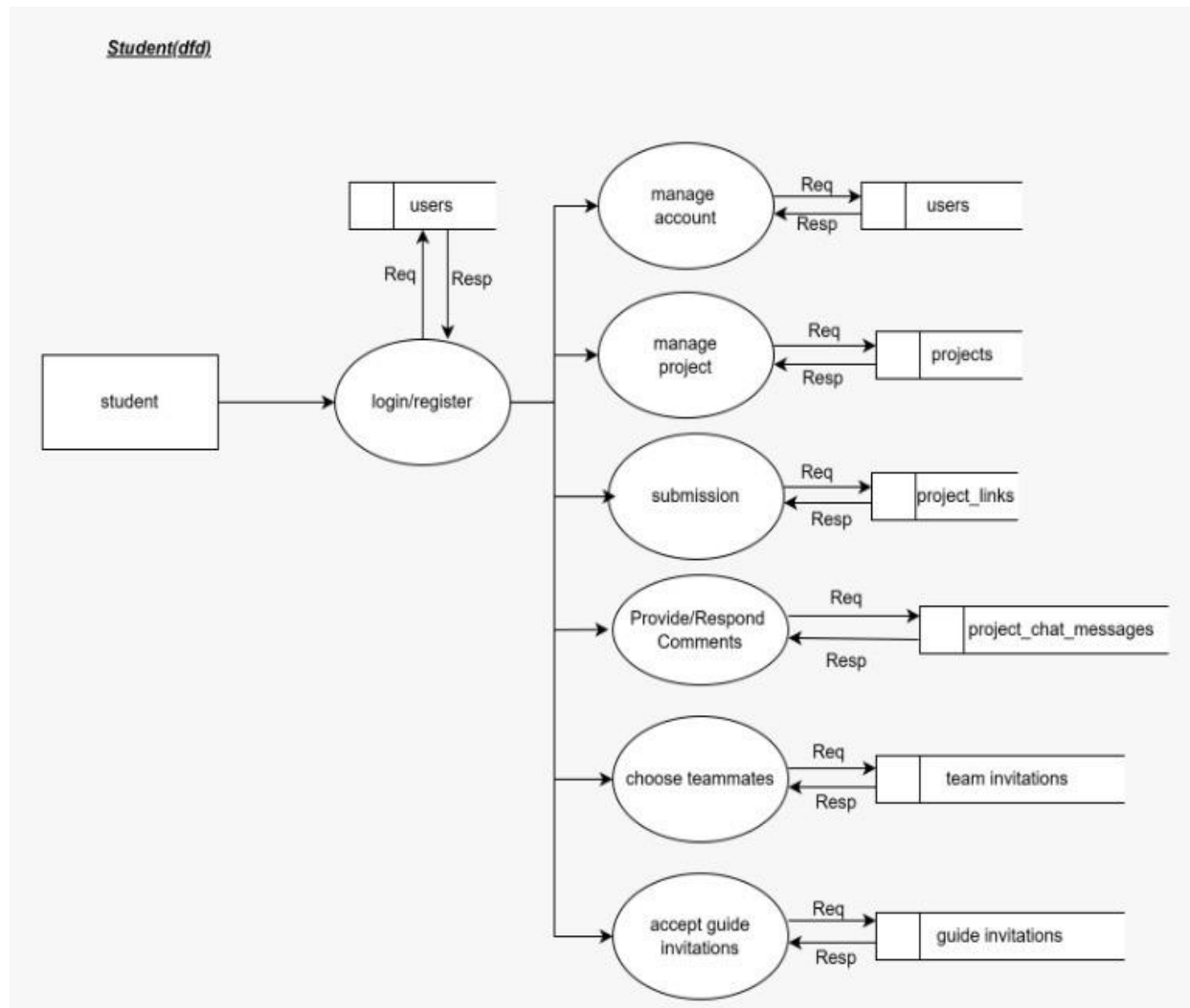
4.2.2 DFD LEVEL 1.0

ADMIN



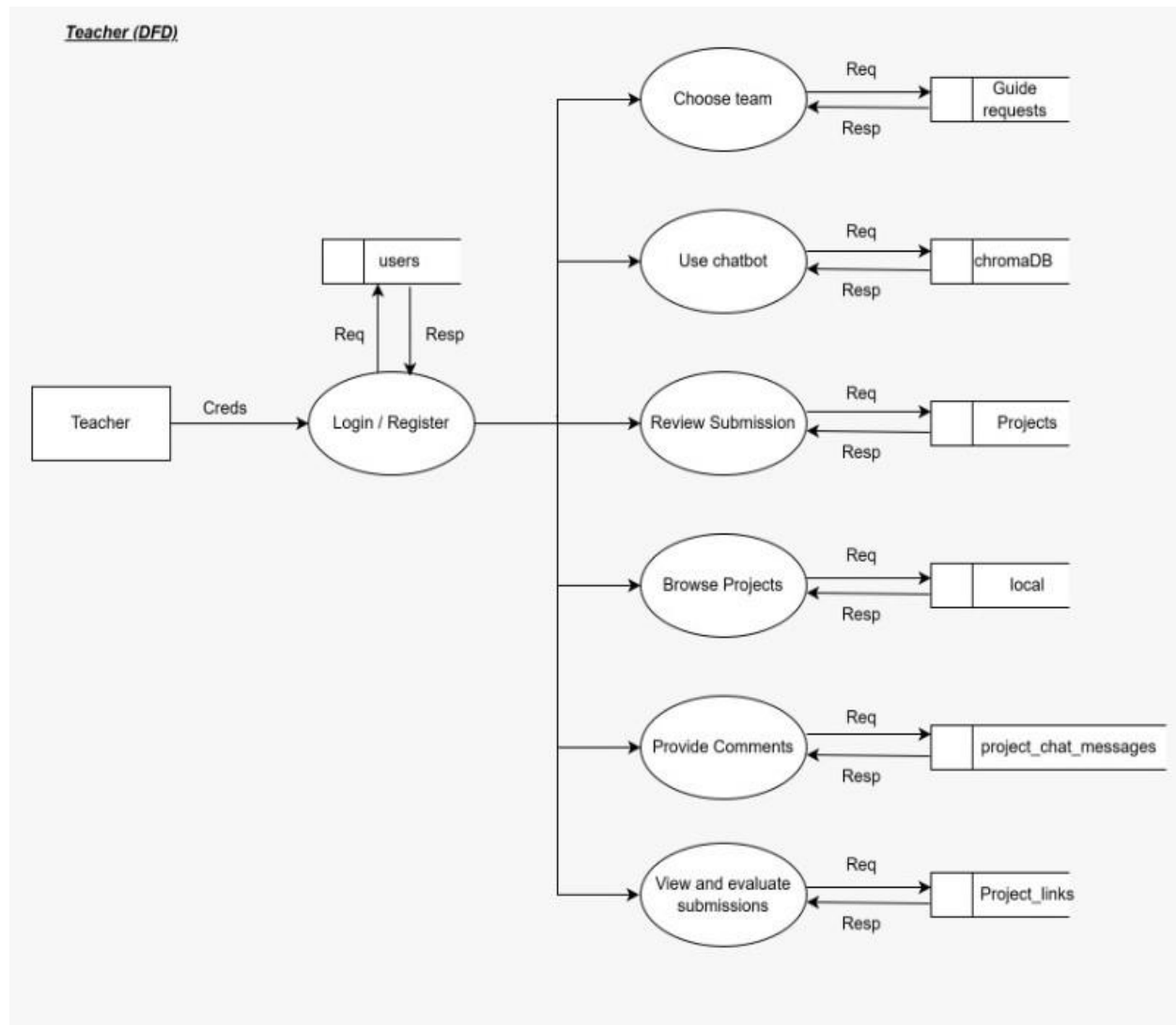
4.2.3 DFD LEVEL 1.1

STUDENT

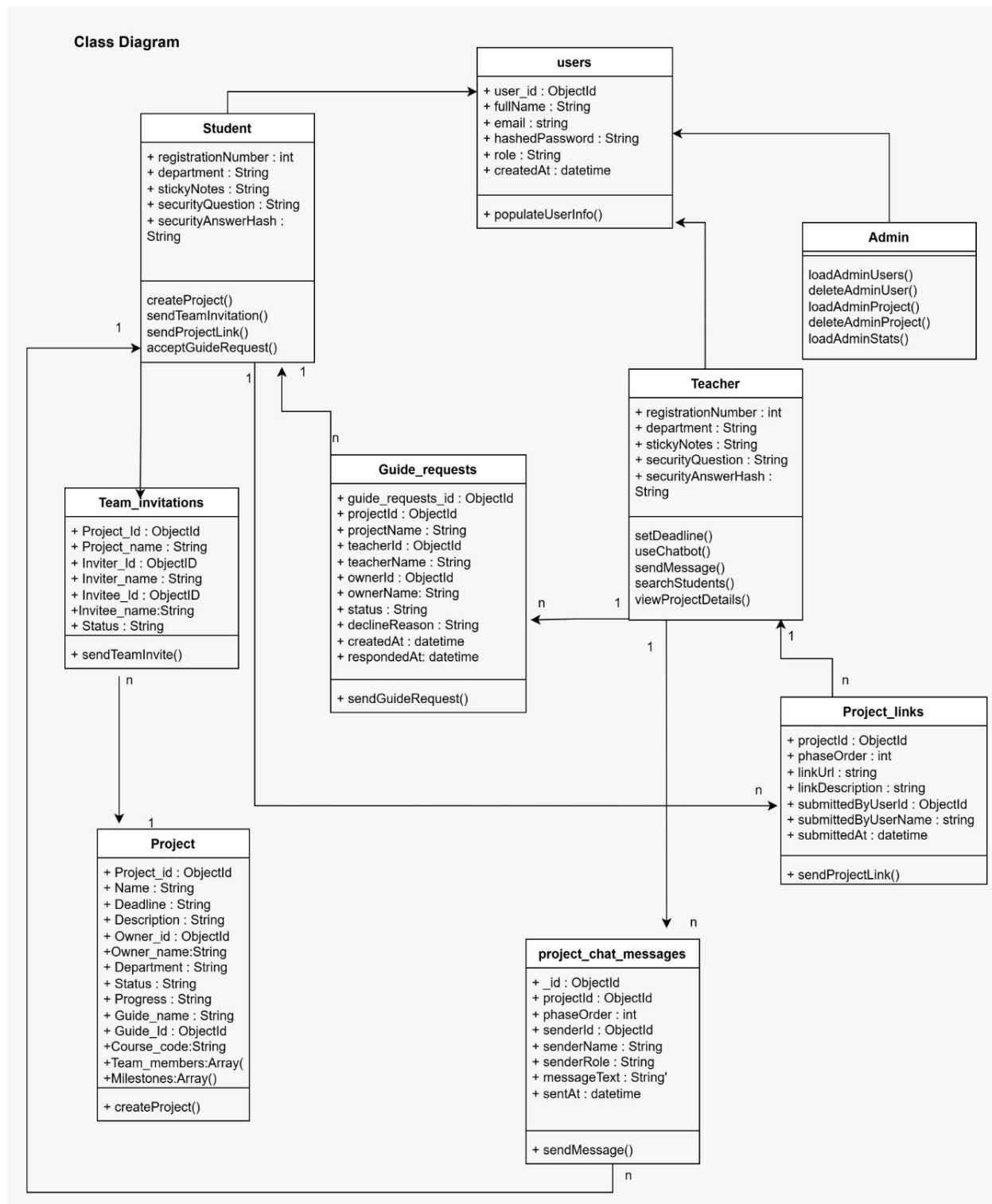


4.2.4 DFD LEVEL 1.2

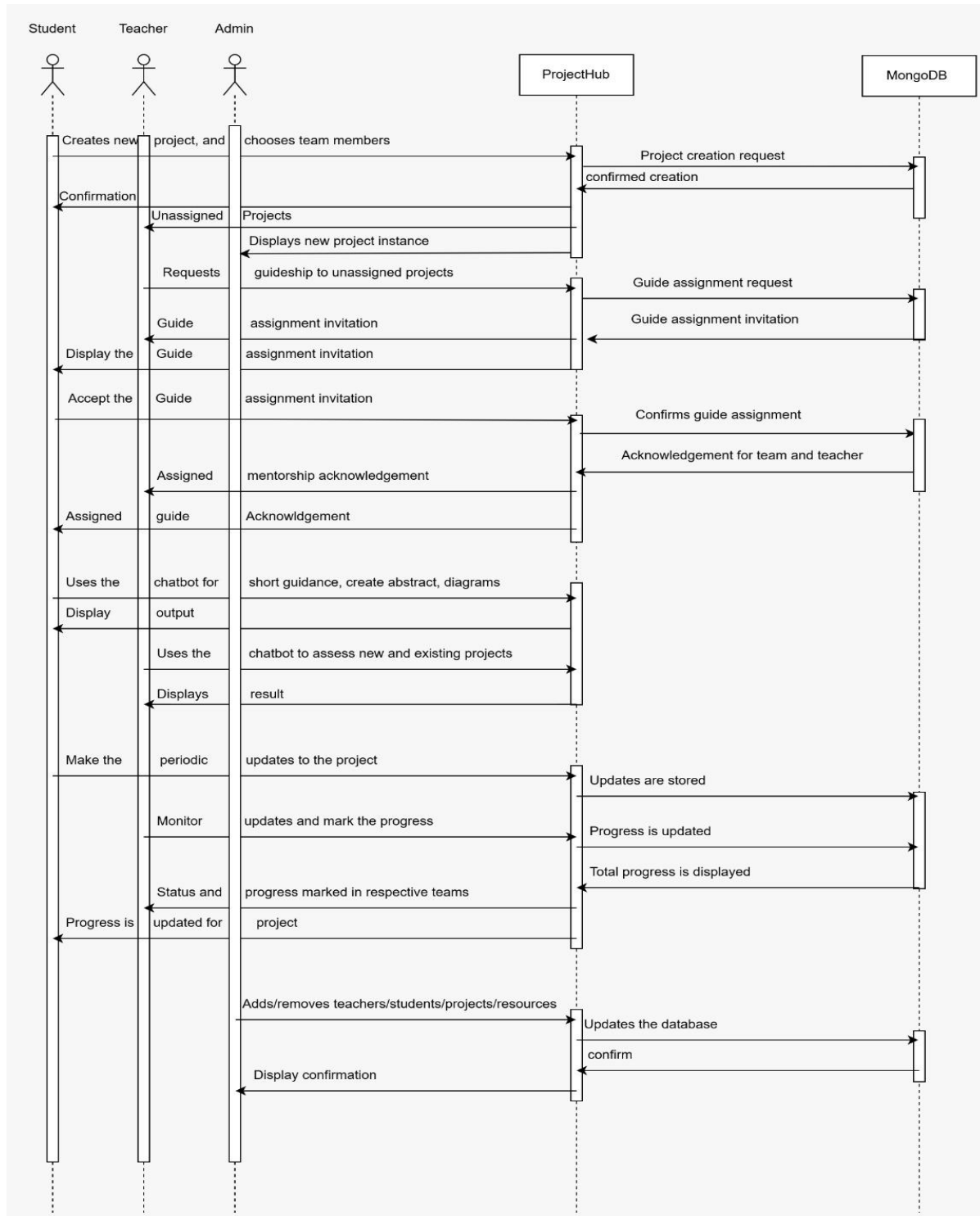
TEACHER



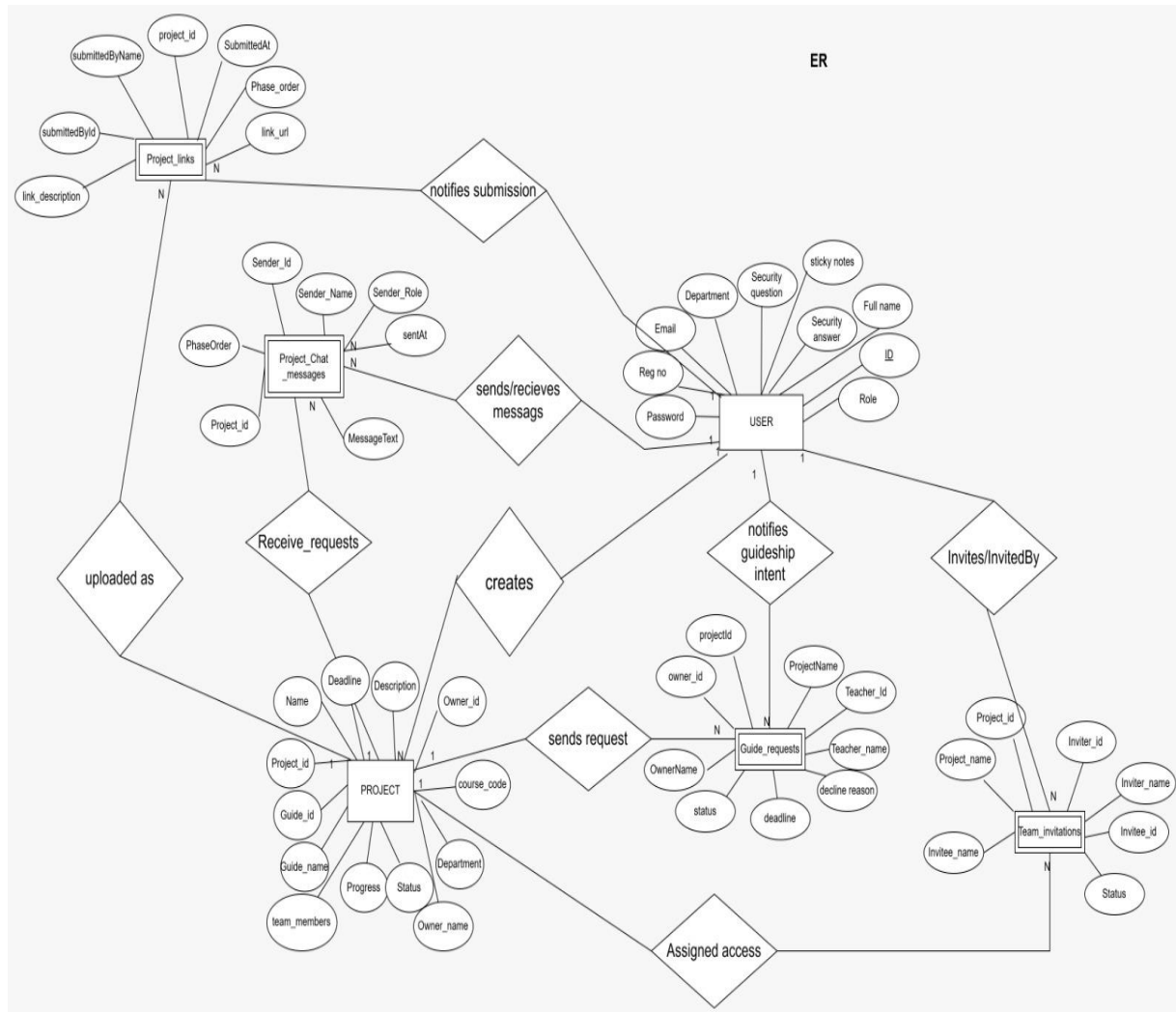
4.3 CLASS DIAGRAM



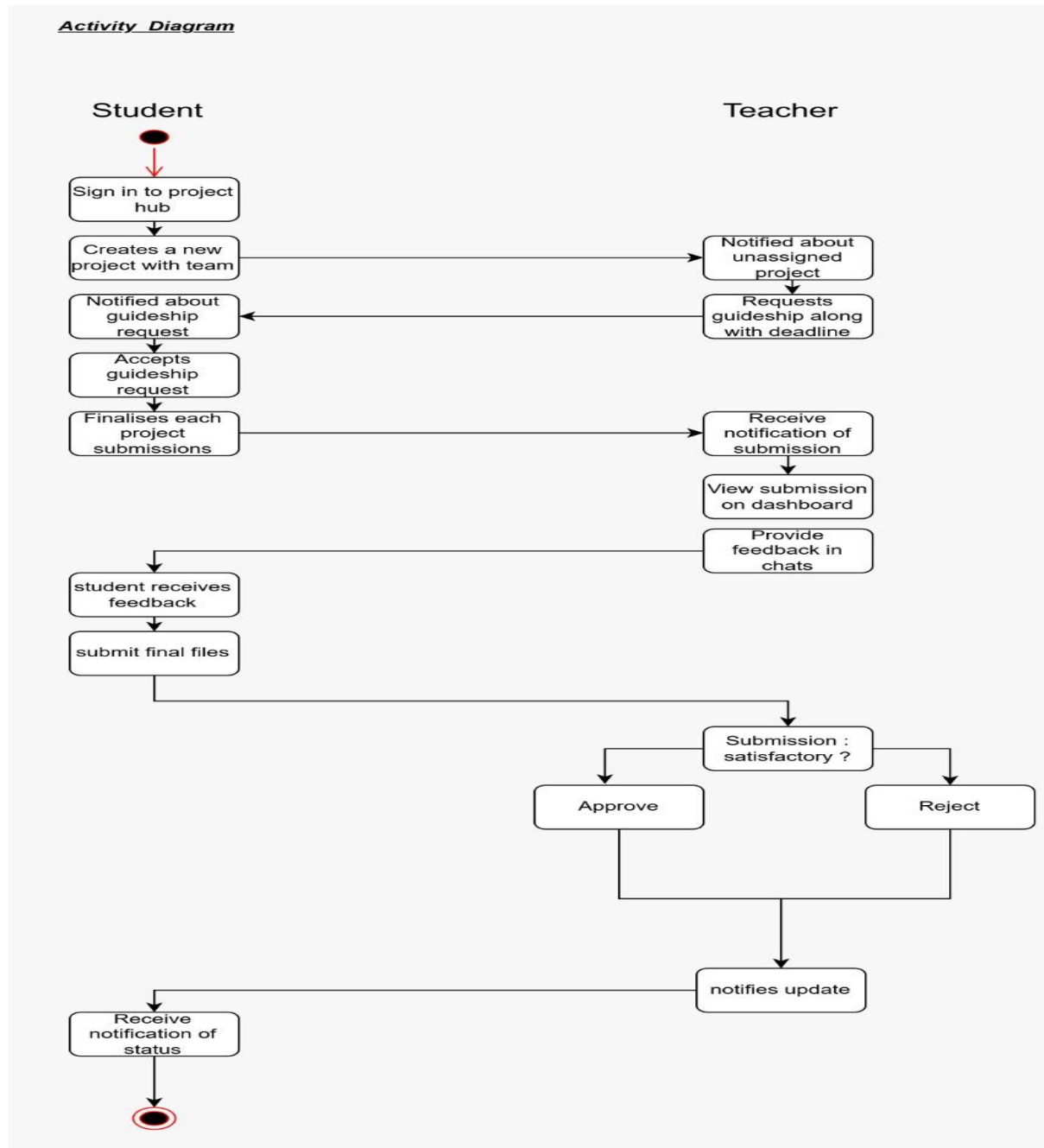
4.4 SEQUENCE DIAGRAM



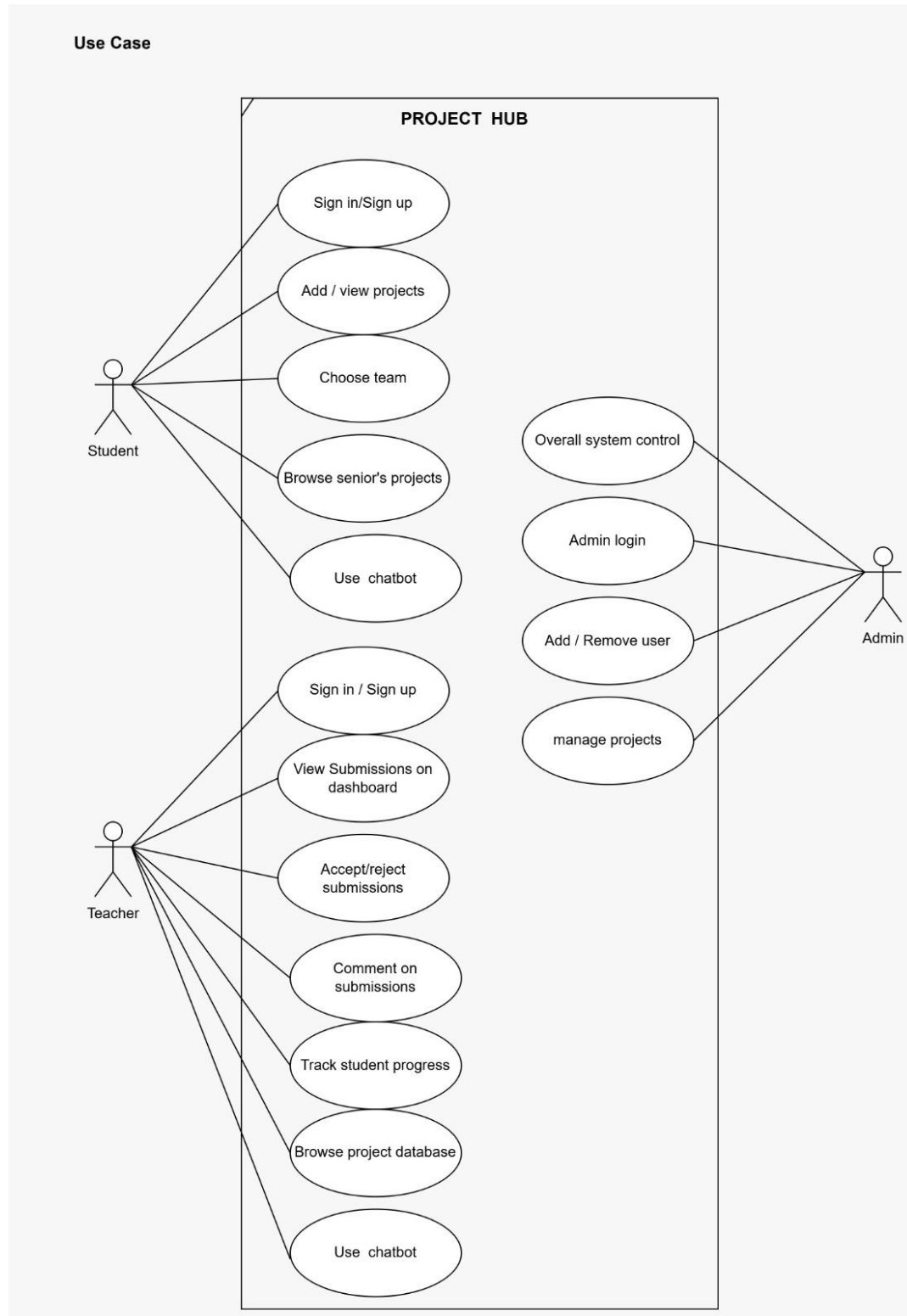
4.5 ER DIAGRAM



4.6 ACTIVITY DIAGRAM



4.7 USE CASE DIAGRAM



**SYSTEM IMPLEMENTATION,
TESTING
&
MAINTENANCE**

5.1 SYSTEM IMPLEMENTATION

We are implementing the system in the Fast API Framework. Fast API is a modern, high-performance web framework for building APIs with Python based on standard Python type hints.

5.2 SYSTEM TESTING

System testing is a level of testing that validates the complete and fully integrated software product. The purpose of a system test is to evaluate the end-to-end system specifications. Usually, the software is only one element of a larger computer-based system. Ultimately, the software interfaces with other software/hardware systems. System Testing is actually a series of different tests whose sole purpose is to exercise the full computer-based system.

Two Category of Software Testing

- Black Box Testing
- White Box Testing

System test falls under the **black box testing** category of software testing.

White box testing is the testing of the internal workings or code of a software application. In contrast, black box or System Testing is the opposite. System test involves the external workings of the software from the user's perspective.

System Testing involves testing the software code for following

- Testing the fully integrated applications including external peripherals in order to check how components interact with one another and with the system as a whole. This is also called an End-to-End testing scenario.
- Verify thorough testing of every input in the application to check for desired outputs.
- Testing of the user's experience with the application.

That is a very basic description of what is involved in system testing. You need to build detailed test cases and test suites that test each aspect of the application as seen from the outside without looking at the actual source code.

5.2.1 UNIT TESTING

Unit testing performed on each module or block of code during development. The programmer who writes the code normally does unit testing. It focuses on the smallest unit of software design. In this we test an individual unit or group of inter related units. Programmers often do it by using sample input and observing its corresponding outputs. The main aim is to isolate each unit of the system to identify, analyse and fix the defects.

ADVANTAGES:

- Reduces Defects in the newly developed features or reduces bugs when changing the existing functionality.
- Reduces Cost of Testing as defects are captured in a very early phase.

- Improves design and allows better refactoring of code.
- Unit Tests, when integrated with build, give the quality of the build as well.

TECHNIQUES:

- **Black Box Testing** - Using which the user interface, input and output are tested.
- **White Box Testing** - used to test each one of those function's behavior is tested.
- **Grey Box Testing** - Used to execute tests, risks and assessment methods.

5.2.2 INTEGRATION TESTING

The meaning of Integration testing is quite straightforward- Integrate/combine the unit tested module one by one and test the behavior as a combined unit. The main function or goal of this testing is to test the interfaces between the units/modules. We normally do Integration testing after “Unit testing”. Once all the individual units are created and tested, we start combining those “Unit Tested” modules and start doing the integrated testing. The main function or goal of this testing is to test the interfaces between the units/modules. The individual modules are first tested in isolation. Once the modules are unit tested, they are integrated one by one, until all the modules are integrated, to check the combinational behavior, and validate whether the requirements are implemented correctly or not. Here we should understand that Integration testing does not happen at the end of the cycle, rather it is conducted simultaneously with the development. So, most of the time all the modules are not actually available to test and here is what the challenge comes to test something which does not exist!

ADVANTAGES:

There are several advantages of this testing and few of them are listed below:

- This testing makes sure that the integrated modules/components work properly.
- Integration testing can be started once the modules to be tested are available. It does not require the other module to be completed for testing to be done, as Stubs and Drivers can be used for the same.
- It detects the errors related to the interface.

5.2.3 SYSTEM TESTING

System Testing (ST) is a black box testing technique performed to evaluate the complete system's compliance against specified requirements. In System testing, the functionalities of the system are tested from an end-to-end perspective. System Testing is usually carried out by a team that is independent of the development team in order to measure the quality of the system unbiased. It includes both functional and Non-Functional testing. Black-box testing treats the software as a "black box", examining functionality without any knowledge of internal implementation, without seeing the source code. The testers are only aware of what the software is supposed to do, not how it does it. Black-box testing methods include: equivalence partitioning, boundary value analysis, all-pairs testing, state transition tables, decision table testing, fuzz testing, model-based testing, use case testing, exploratory testing and specification-based testing. Specification-based testing aims to

test the functionality of software according to the applicable requirements. This level of testing usually requires thorough test cases to be provided to the tester, who then can simply verify that for a given input, the output value (or behavior), either "is" or "is not" the same as the expected value specified in the test case. Test cases are built around specifications and requirements, i.e., what the application is supposed to do. It uses external descriptions of the software, including specifications, requirements, and designs to derive test cases. These tests can be functional or non-functional, though usually functional. Specification-based testing may be necessary to assure correct functionality, but it is insufficient to guard against complex or high-risk situations. One advantage of the black box technique is that no programming knowledge is required. Whatever biases the programmers may have had, the tester likely has a different set and may emphasize different areas of functionality. On the other hand, black-box testing has been said to be "like a walk in a dark labyrinth without a flashlight. "Because they do not examine the source code, there are situations when a tester writes many test cases to check something that could have been tested by only one test case, or leaves some parts of the program untested.

5.2.4 TEST REPORT

Test Report is needed to reflect testing results in a formal way, which gives an opportunity to estimate testing results quickly. It is a document that records data obtained from an evaluation experiment in an organized manner, describes the environmental or operating conditions, and shows the comparison of test results with test objectives.

5.3 SYSTEM MAINTENANCE

An integral part of software is the maintenance one, which requires an accurate maintenance plan to be prepared during the software development. It should specify how users will request modifications or report problems. The budget should include resource and cost estimates. A new decision should be addressed for the development of every new system feature and its quality objectives. The software maintenance, which can last for 5–6 years (or even decades) after the development process, calls for an effective plan which can address the scope of software maintenance, the tailoring of the post-delivery/deployment process, the designation of who will provide maintenance, and an estimate of the life-cycle costs. The selection of proper enforcement of standards is the challenging task right from the early stage of software engineering which has not got definite importance by the concerned stakeholders.

SAMPLE CODE

main.py

```
from fastapi import FastAPI, HTTPException, Depends
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import JSONResponse
from pydantic import BaseModel
from typing import Optional, List

from database import get_db
from auth import verify_token, hash_password, check_password
from models import User, Project, ChatMessage

app = FastAPI(title="Project Management + AI Chat System")

# -----
# CORS
# -----

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)
```



```
# -----  
  
# MODELS  
  
# -----  
  
class SignupRequest(BaseModel):  
    username: str  
    password: str  
    email: str  
  
  
class LoginRequest(BaseModel):  
    username: str  
    password: str  
  
  
class ProjectRequest(BaseModel):  
    title: str  
    description: str  
  
  
class ChatRequest(BaseModel):  
    project_id: int  
    message: str
```

```
# -----  
  
# AUTHENTICATION  
  
# -----  
  
@app.post("/signup")  
def signup(data: SignupRequest, db=Depends(get_db)):  
    if db.query(User).filter(User.username == data.username).first():  
        raise HTTPException(400, "Username already exists")  
  
    new_user = User(  
        username=data.username,  
        email=data.email,  
        password=hash_password(data.password)  
    )  
    db.add(new_user)  
    db.commit()  
    return {"message": "User created successfully"}  
  
  
@app.post("/login")  
def login(data: LoginRequest, db=Depends(get_db)):
```

```
user = db.query(User).filter(User.username == data.username).first()

if not user or not check_password(data.password, user.password):

    raise HTTPException(401, "Invalid credentials")


token = verify_token(user.id, create=True)

return {"token": token, "username": user.username}


# -----
# PROJECT MANAGEMENT
# -----


@app.post("/project/create")
def create_project(data: ProjectRequest, user=Depends(verify_token), db=Depends(get_db)):

    project = Project(

        title=data.title,

        description=data.description,

        user_id=user.id

    )

    db.add(project)

    db.commit()

    return {"message": "Project created", "project_id": project.id}
```

```
@app.get("/project/list")
```

```
def list_projects(user=Depends(verify_token), db=Depends(get_db)):
```

```
    projects = db.query(Project).filter(Project.user_id == user.id).all()
```

```
    return {"projects": projects}
```

```
@app.delete("/project/{project_id}")
```

```
def delete_project(project_id: int, user=Depends(verify_token), db=Depends(get_db)):
```

```
    project = db.query(Project).filter(
```

```
        Project.id == project_id,
```

```
        Project.user_id == user.id
```

```
    ).first()
```

```
    if not project:
```

```
        raise HTTPException(404, "Project not found")
```

```
    db.delete(project)
```

```
    db.commit()
```

```
    return {"message": "Project deleted"}
```

```
# -----
```

```
# CHAT + AI INTEGRATION
```

```
# -----
```

```
@app.post("/chat/send")
```

```
def send_chat_message(data: ChatRequest, user=Depends(verify_token), db=Depends(get_db)):
```

```
    # Save user message
```

```
    msg = ChatMessage(  
project_id=data.project_id,  
sender_id=user.id,  
    message=data.message  
    )
```

```
db.add(msg)
```

```
db.commit()
```

```
    # Call AI (abstracted)
```

```
ai_reply = generate_ai_response(data.message, project_id=data.project_id)
```

```
    # Save AI response
```

```
ai_msg = ChatMessage(  
project_id=data.project_id,  
sender_id=None,  
    message=ai_reply  
    )
```

```
db.add(ai_msg)
```

```
db.commit()
```

```
    return {"reply": ai_reply}
```

```
# -----
```

```
# FETCH CHAT HISTORY
```

```
# -----
```

```
@app.get("/chat/{project_id}")
```

```
def get_chat_history(project_id: int, user=Depends(verify_token), db=Depends(get_db)):
```

```
    history = db.query(ChatMessage).filter(
```

```
    ChatMessage.project_id == project_id
```

```
).order_by(ChatMessage.timestamp.asc()).all()
```

```
    return {"messages": history}
```

```
# -----
```

```
# AI LOGIC
```

```
# -----
```

```
def generate_ai_response(message: str, project_id: Optional[int] = None):
```

```
    """
```

```
    Minimal placeholder.
```

```
    Real logic handled in RAG18.py
```

```
    """
```

```
    return f"AI response to: {message}"
```

```
# -----
```

```
# ROOT ENDPOINT
```

```
# -----
```

```
@app.get("/")
```

```
def root():
```

```
    return {"message": "Backend running successfully"}
```

RAG.py

```
import os, shutil, json, asyncio, uvicorn
```

```
from pathlib import Path
```

```
from typing import List, Dict, Any
```

```
from pydantic import BaseModel
```

```
from fastapi import FastAPI, HTTPException
```

```
from fastapi.responses import JSONResponse, StreamingResponse
```

```
from fastapi.staticfiles import StaticFiles
```

```
from langchain_community.document_loaders import PyPDFLoader, TextLoader

from langchain_text_splitters import RecursiveCharacterTextSplitter

from langchain_chroma import Chroma

from langchain.embeddings.base import Embeddings


# Check imports

try:

    from google import genai

    from langchain_ollama import ChatOllama

    from langchain_ollama.embeddings import OllamaEmbeddings

    import ollama

except ImportError:

    exit("Required packages missing. Please install google-genai, langchain-ollama, ollama.")


# Paths & Config

SCRIPT_DIR = Path(_file_).resolve().parent

FRONTEND_DIR = "../frontend"

CONFIG_FILE = "rag_config.json"

DEFAULT_DB_PATH = "./chroma_db"

SYSTEM_PROMPT = "You are HubMaster. Provide helpful, clear, and engaging responses."


class GeminiEmbeddings(Embeddings):

    """Wrapper for Google GenAI Embeddings"""
```



```
def _init_(self, client, model="text-embedding-004"):
    self.client, self.model = client, model

    def embed_documents(self, texts):
        return [self.client.models.embed_content(model=self.model,
            contents=t).embeddings[0].values for t in texts]

    def embed_query(self, text):
        return self.client.models.embed_content(model=self.model,
            contents=text).embeddings[0].values

class RAGSystem:
    def _init_(self):
        self.config = self.load_config()
        self.client, self.llm, self.vectorstore, self.retriever = None, None, None, None
        self.embeddings = None

    def load_config(self):
        if os.path.exists(CONFIG_FILE): return json.load(open(CONFIG_FILE))
        return {"provider": "gemini", "gemini_model": "gemini-2.5-flash", "rag_docs_path":
            str(SCRIPT_DIR.parent / "Docs"), "vector_db_path": DEFAULT_DB_PATH, "system_prompt":
            SYSTEM_PROMPT}

    def save_config(self, data):
        self.config.update(data)
```

```
with open(CONFIG_FILE, 'w') as f: json.dump(self.config, f, indent=2)

def initialize(self):

    try:

        if self.config["provider"] == "gemini":

            self.client = genai.Client(api_key=self.config["gemini_api_key"])

            self.embeddings = GeminiEmbeddings(self.client)

        elif self.config["provider"] == "ollama":

            self.llm = ChatOllama(model=self.config["ollama_model"],

            base_url=self.config["ollama_endpoint"])

            self.embeddings = OllamaEmbeddings(model=self.config["embedding_model"],

            base_url=self.config["ollama_endpoint"])

            # Load existing vectorstore if available

            if os.path.exists(self.config["vector_db_path"]):

                self.vectorstore = Chroma(persist_directory=self.config["vector_db_path"],

                embedding_function=self.embeddings)

            self.retriever = self.vectorstore.as_retriever(search_kwargs={"k": 3})

            return True

        except Exception as e:

            print(f"Init Error: {e}")

            return False
```

```
def build_db(self, files, force=False):

    if force and os.path.exists(self.config["vector_db_path"]):
shutil.rmtree(self.config["vector_db_path"], ignore_errors=True)

self.vectorstore = None


docs = []

for f in files:

    try:

        loader = PyPDFLoader(f) if f.endswith('.pdf') else TextLoader(f, encoding='utf-8')
docs.extend(loader.load())

        except: pass # Skip bad files


    if not docs: return False


    chunks = RecursiveCharacterTextSplitter(chunk_size=800,
chunk_overlap=100).split_documents(docs)

self.vectorstore = Chroma.from_documents(chunks, self.embeddings,
persist_directory=self.config["vector_db_path"])

self.retriever = self.vectorstore.as_retriever(search_kwargs={"k": 3})

    return True


async def stream_chat(self, query, mode):

    prompt = f'{{self.config.get('system_prompt', SYSTEM_PROMPT)}}\n\n'
```

```
        if mode == "rag" and self.retriever:

            docs = self.retriever.invoke(query)

            ctx = "\n".join([d.page_content for d in docs])

            prompt += f"Context:\n{ctx}\n\nQuestion: {query}"

        else:

            prompt += f"User: {query}"

    try:

        if self.config["provider"] == "gemini":

            async for chunk in
self.client.models.generate_content_stream(model=self.config["gemini_model"],
contents=prompt):

                yield chunk.text

    elif self.config["provider"] == "ollama":

        async for chunk in self.llm.astream(prompt):

            yield chunk.content

    except Exception as e: yield f"Error: {str(e)}"

# FastAPI Setup

app = FastAPI()

rag = RAGSystem()
```

```
class ConfigReq(BaseModel): provider: str; gemini_api_key: str = ""; ollama_endpoint: str = "";  
gemini_model: str = ""; ollama_model: str = ""; embedding_model: str = ""; rag_docs_path: str =  
""; vector_db_path: str = ""; system_prompt: str = ""
```

```
class InitReq(BaseModel): config: Dict[str, Any]; selected_files: List[str]; force_rebuild: bool
```

```
class ChatReq(BaseModel): query: str; mode: str
```

```
@app.get("/api/config")
```

```
async def get_config(): return rag.load_config()
```

```
@app.post("/api/config")
```

```
async def save_config(req: ConfigReq):
```

```
    rag.save_config(req.dict())
```

```
    return {"status": "success"}
```

```
@app.get("/api/ollama-models")
```

```
async def get_ollama_models():
```

```
    try:
```

```
        models = ollama.Client(host=rag.config.get("ollama_endpoint")).list().models
```

```
        return {"models": [m.model for m in models]}
```

```
    except: return {"models": []}
```

```
@app.get("/api/documents")
```

```
async def get_docs():
```

```
files = []

path = rag.config.get("rag_docs_path", str(SCRIPT_DIR.parent / "Docs"))

for r, _, f_list in os.walk(path):

    for f in f_list:

        if f.endswith((''.txt', '.pdf', '.md')):

files.append({"name": f, "path": os.path.join(r, f), "folder": os.path.relpath(r, path)})

return {"files": files}


@app.get("/api/resources")

async def get_resources():

guide_dir = SCRIPT_DIR.parent / "frontend" / "README Guide"

if not guide_dir.exists(): return {"files": []}

return {"files": sorted([f.name for f in guide_dir.iterdir() if f.suffix == '.txt'])}


@app.post("/api/initialize")

async def initialize(req: InitReq):

rag.save_config(req.config)

if not rag.initialize(): raise HTTPException(500, "Provider Init Failed")

if req.selected_files:

    if not rag.build_db(req.selected_files, req.force_rebuild): raise HTTPException(500, "DB

Build Failed")

return {"status": "success", "message": "Initialized"}
```

```
@app.post("/api/chat")

async def chat(req: ChatReq):

    return StreamingResponse(rag.stream_chat(req.query, req.mode), media_type="text/event-
stream")

app.mount("/", StaticFiles(directory=FRONTEND_DIR, html=True), name="static")

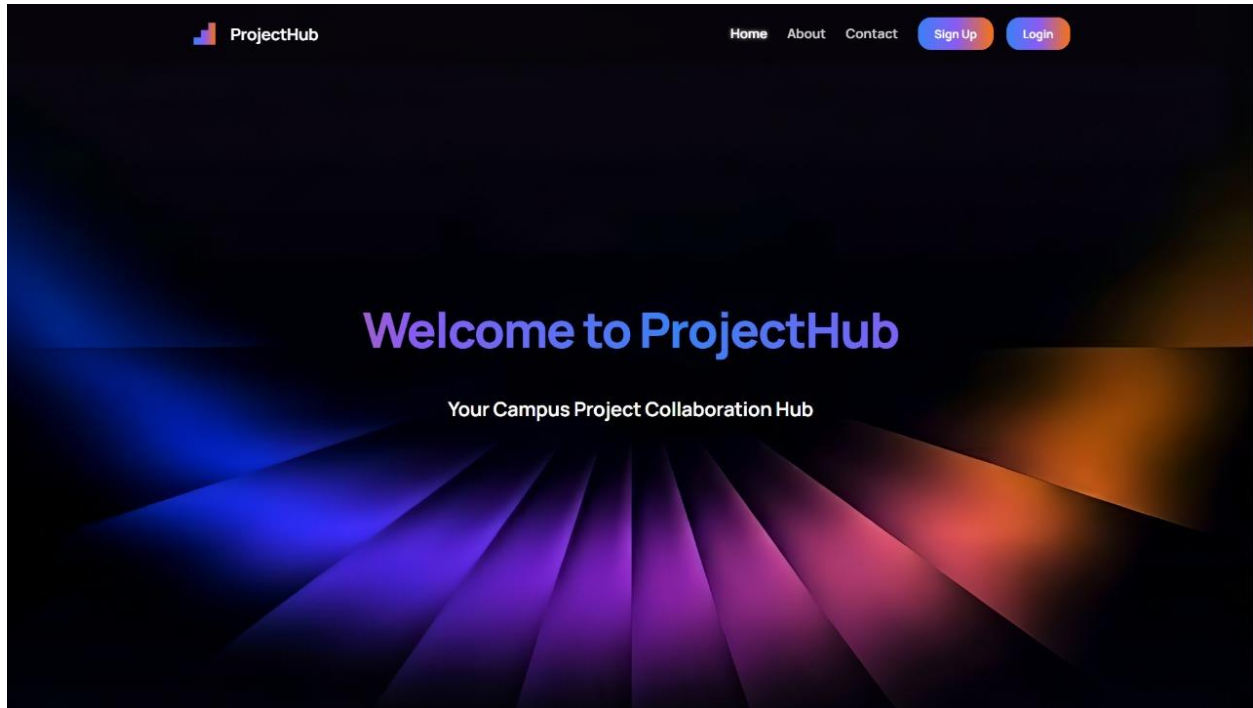
if __name__ == "__main__":

    if not os.path.exists(FRONTEND_DIR): exit("Frontend not found.")

uvicorn.run(app, host="127.0.0.1", port=8000)
```

SCREENSHOTS

i. HOME



ii. USER REGISTRATION

The screenshot shows the ProjectHub User Registration page. At the top, there is a navigation bar with a back arrow, the ProjectHub logo, and links for Home, About, Contact, Sign Up, and Login. The main content area features a large, colorful background with a perspective effect of a road or hallway. The text "Welcome to ProjectHub!" is prominently displayed in the center, with the subtitle "Let's get your profile set up." below it. Below the subtitle, there is a registration form with the following fields: Full Name (text input), Registration Number (text input with example "e.g., 2023-CS-123"), Email (text input), Password (text input with "Min. 8 characters" requirement and a strength indicator), Department (dropdown menu), User Role (dropdown menu with "Student" selected), Security Question (dropdown menu), and Security Answer (text input with "Your answer (will be saved in caps)" requirement). A "Complete Setup" button is located at the bottom of the form.

iii. USER LOGIN

 ProjectHub

Home About Contact [Sign Up](#) [Login](#)

Log In to ProjectHub

Email

noel@hub

Password

.....



[Forgot password?](#)

[Log In](#)

Don't have an account? [Sign up](#)

iv. STUDENT DASHBOARD

 ProjectHub

[Dashboard](#)
[Projects](#)
[My Team](#)
[Resources](#)
[Settings](#)

Welcome, Naveen!

Here's an overview of your campus activity.

Current Projects

Loading projects...

[Manage All Projects](#)

Upcoming Deadlines

New project by akash for anoop sir
Dec 25, 2025

MCA project 2
Dec 31, 2025

Team Invitations

You have no new invitations.

[View All](#)

Guide Requests

Request from Anoop S
For: sample project header from jayal

[View All](#)

Recent Comments

Anoop S
"hello - 2"

Anoop S
"hello"

Anoop S
"this is my 7th text"

[+ Add New Project](#)

Calendar

December 2025

Sun	Mon	Tue	Wed	Thu	Fri	Sat
	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30	31			

Naveen N
Student
[Logout](#)



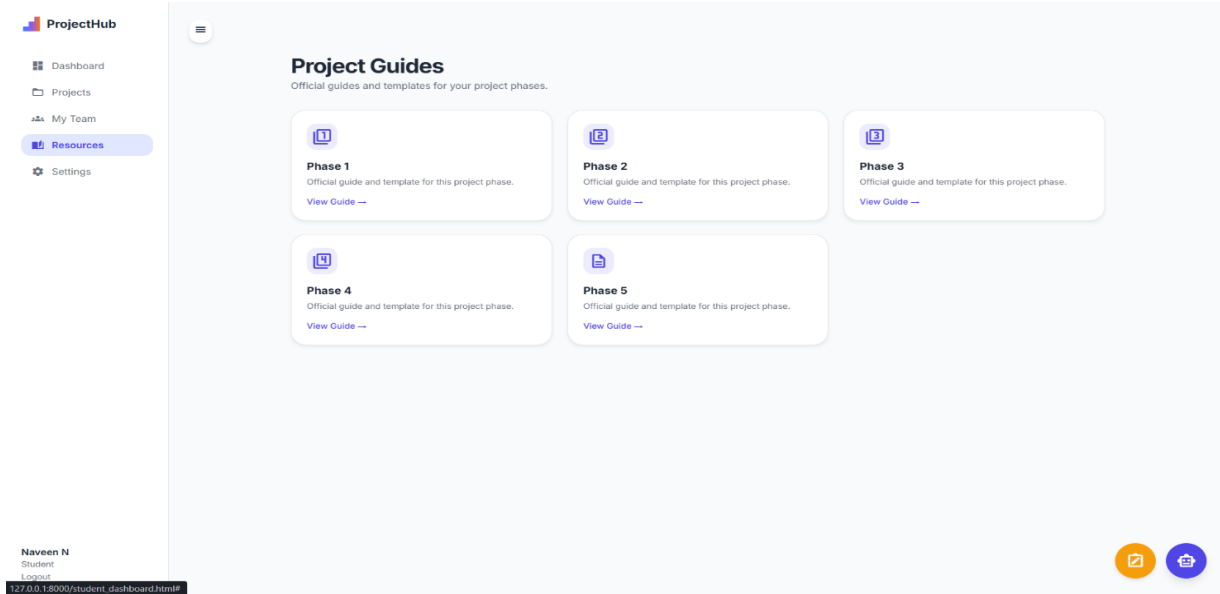
v. STUDENT PROJECTS

The screenshot shows the 'My Projects' dashboard in ProjectHub. The left sidebar contains navigation links: Dashboard, Projects (selected), My Team, Resources, and Settings. The main content area is titled 'My Projects' with a subtitle 'View and manage all your projects in one place.' and an '+ Add New Project' button. Below the title is a search bar and tabs for 'All', 'Active', and 'Past' projects. Five project cards are displayed, each showing the project name, course, progress bar, and a 'View Details' link. The projects are: 'election voting' (25% progress), 'tdhtfgilgijy jhygiyu' (0% progress), 'sample project hello from joyal' (0% progress), 'MCA project 2' (100% progress), and 'New project by akash for anoop sir' (0% progress). The bottom left shows the user 'Naveen N' (Student) with a 'Logout' link. The bottom right has two circular icons: a yellow one with a document and a blue one with a gear.

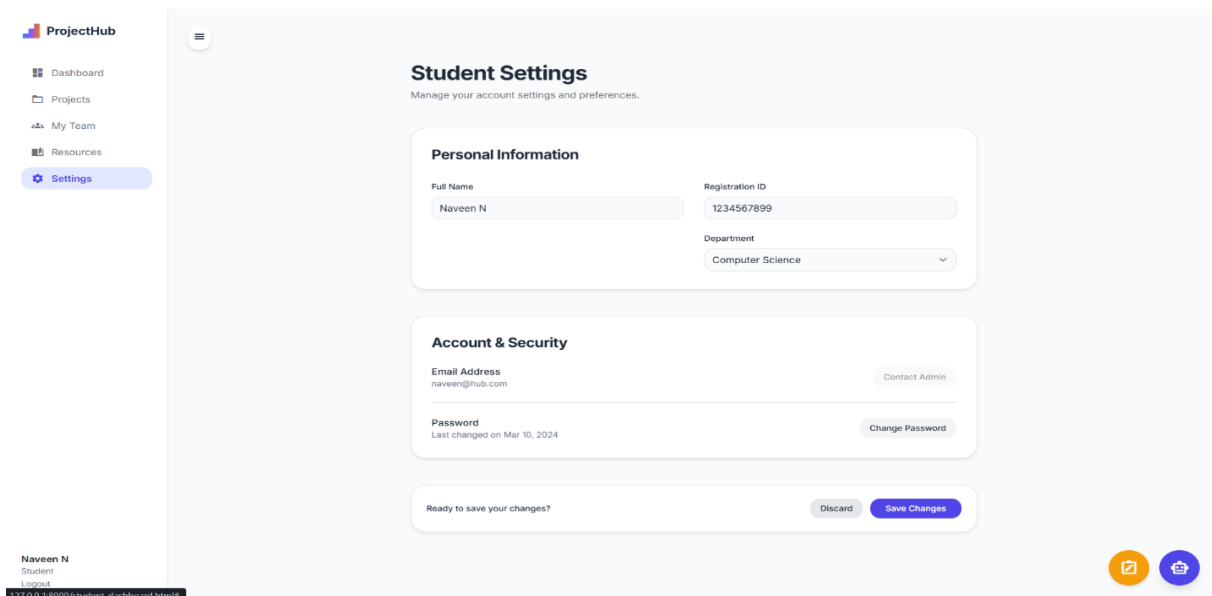
vi. STUDENT MY TEAM & INVITATIONS

The screenshot shows the 'My Team & Invitations' dashboard in ProjectHub. The left sidebar is the same as in the previous screenshot. The main content area is titled 'My Team & Invitations' with a subtitle 'View and manage all your team and guides in one place.' Below the title is a search bar and tabs for 'All', 'Active', and 'Past' teams. The 'Active Teams' section displays five project cards, each with a 'View Project' button. The projects are: 'election voting', 'tdhtfgilgijy jhygiyu', 'sample project hello from joyal', 'MCA project 2', and 'New project by akash for anoop sir'. The 'Pending Invitations' section is divided into two parts: 'Project Invitations' (showing 'You have no new project invitations.') and 'Guide Requests' (showing a request from 'Anoop S' to guide the project 'sample project hello from joyal' with 'Accept' and 'Decline' buttons). The bottom left shows the user 'Naveen N' (Student) with a 'Logout' link. The bottom right has the same two circular icons as the previous screenshot.

vii. STUDENT RESOURCES



viii. STUDENT SETTINGS



ix. STUDENT PROJECT CREATION

The screenshot shows the 'Create New Project' form in the ProjectHub interface. The form is titled 'Create New Project' and includes a progress bar with three steps: 1. DETAILS, 2. TEAM, and 3. GUIDE. The first step, 'DETAILS', is active. The form contains three input fields: 'Project Name' (with a placeholder 'e.g., Quantum Computing Simulation'), 'Project Description' (with a placeholder 'Briefly describe your project...'), and 'Course Code (Optional)' (with a placeholder 'e.g., COMP 401'). At the bottom right of the form are 'Cancel' and 'Next' buttons. The left sidebar shows the ProjectHub logo and navigation links: Dashboard, Projects, My Team, Resources, and Settings. The bottom left shows the user's name 'Naveen N', role 'Student', and a 'Logout' link. The bottom right has two circular icons: a yellow one with a document and a blue one with a person.

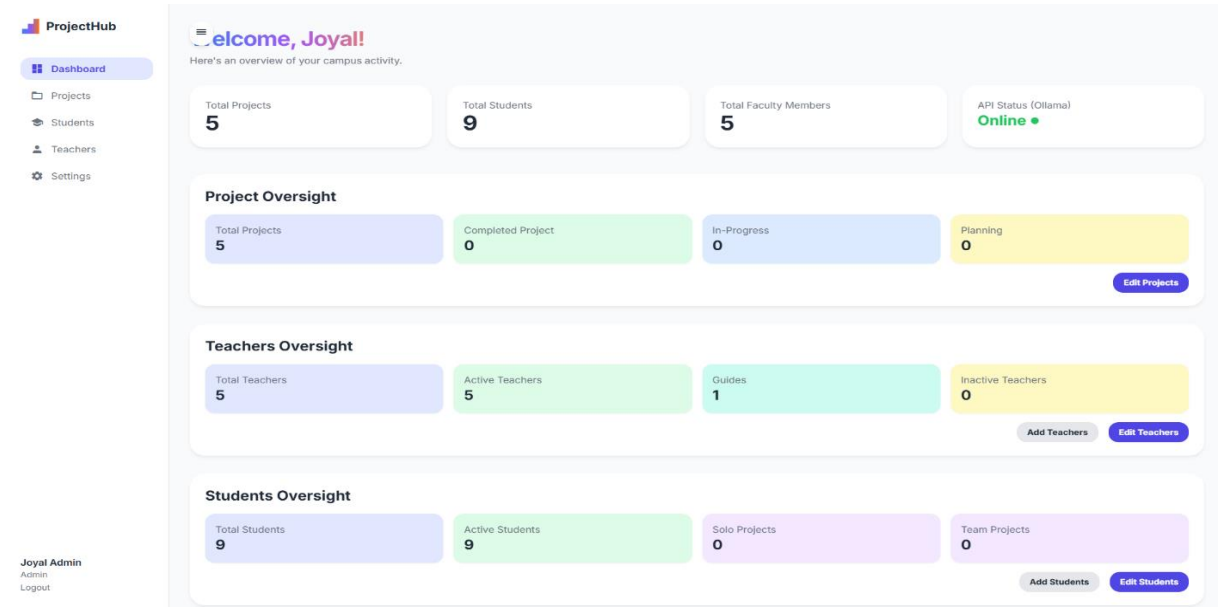
x. STUDENT PROJECT UPLOAD

The screenshot shows the 'Abstract Creation' page in the ProjectHub interface. The page is titled 'Abstract Creation' and includes a 'Back to Project Details' link. Below the title is a 'Phase Checklist' section with the text 'Ensure your Phase 1 submission (Abstract) is complete.' and a list of items: 'Clear Project Title', 'Defined Objective and Scope', 'Chosen Technology Stack', and 'Final Abstract (~250 words)'. Below the checklist is a note: 'Upload a single file (PDF or DOCX) with your abstract and plan.' To the right of the checklist is a 'Submit a Link' section with a 'File Link (Google Drive, etc.)' input field (placeholder 'https://...'), a 'Description' input field (placeholder 'e.g., "Final Report PDF"'), and a 'Submit Link' button. Below the 'Submit a Link' section is a 'Submitted Files' section showing a file named 'dcdsvcds' submitted by 'Naveen N' on 'Dec 2, 2025' with a link 'https://chat.deepseek.co m'. The left sidebar shows the ProjectHub logo and navigation links: Dashboard, Projects, My Team, Resources, and Settings. The bottom left shows the user's name 'Naveen N', role 'Student', and a 'Logout' link. The bottom right has two circular icons: a yellow one with a document and a blue one with a person.

xi. STUDENT CHAT



xii. ADMIN DASHBORAD



xiii. ADMIN MANAGE PROJECTS

ProjectHub

- Dashboard
- Projects**
- Students
- Teachers
- Settings

Manage All Projects

Search projects (name, owner, guide, star) Reset

Project Name	Owner	Guide	Status	Due Date	Actions
MCA project 2	Naveen N	Anoop S	in_progress	Dec 31, 2025	✎ ✖
New project by akash for anoop sir	Akash Manikandan	Anoop S	in_progress	Dec 25, 2025	✎ ✖
election voting	Akash Manikandan	Anoop S	in_progress	Nov 29, 2025	✎ ✖
sample project hello from joyal	Naveen N	N/A	not_started	N/A	✎ ✖
tdhtfgilgy jhygyu	Naveen N	N/A	not_started	N/A	✎ ✖

Joyal Admin
Admin
Logout
127.0.0.1:8000/admin_dashboard.html#

+ Add Student

xiv. ADMIN MANAGE STUDENTS

ProjectHub

- Dashboard
- Projects
- Students**
- Teachers
- Settings

Manage Students

Search students (name, email, ID, dept...) Reset

Student Name	Email	Reg. Number	Department	Join Date	Actions
Naveen N	naveen@hub.com	1234567899	Computer Science	Nov 6, 2025	✎ ✖
Akash Manikandan	akash@hub.com	1234567898	Computer Science	Nov 6, 2025	✎ ✖
Nandhu Krishna	nandhu@hub.com	1234567897	Computer Science	Nov 6, 2025	✎ ✖
Abhishek T U	abhishek@hub.com	123456789678	Computer Science	Nov 6, 2025	✎ ✖
Sahil T	sahil@hub.com	123456784567	Electrical Engineering	Nov 6, 2025	✎ ✖
Anjali S	anjali@hub.com	12345678987654	Computer Science	Nov 6, 2025	✎ ✖
Sakshi K	sakshi@hub.com	12345678953	Computer Science	Nov 6, 2025	✎ ✖
Varsha Siju	varsha@hub.com	123456787654	Electrical Engineering	Nov 6, 2025	✎ ✖
Nived	nived@hub.com	1234567876545	Electrical Engineering	Nov 6, 2025	✎ ✖

Joyal Admin
Admin
Logout
127.0.0.1:8000/admin_dashboard.html#

+ Add Student

xv. ADMIN MANAGE TEACHER

The screenshot shows the 'Manage Teachers' page in the ProjectHub admin interface. On the left is a sidebar with navigation links: Dashboard, Projects, Students, Teachers (selected), and Settings. The main content area has a title 'Manage Teachers' and an '+ Add Teacher' button. Below the title is a search bar with the placeholder 'Search teachers (name, email, id, dept...)' and a 'Reset' button. A table lists five teachers with columns for Teacher Name, Email, Reg. Number, Department, Join Date, and Actions. The teachers listed are Anoop S, Radhika B, Divya G, Joseph T, and Neethu Ganesh. At the bottom left, the user 'Joyal Admin' is logged in.

Teacher Name	Email	Reg. Number	Department	Join Date	Actions
Anoop S	anoop@hub.com	98765432198	Computer Science	Nov 6, 2025	
Radhika B	radhika@hub.com	9876543212345	Computer Science	Nov 6, 2025	
Divya G	divya@hub.com	123456789765	Computer Science	Nov 6, 2025	
Joseph T	joseph@hub.com	9876543217654	Electrical Engineering	Nov 6, 2025	
Neethu Ganesh	neethu@hub.com	9876543218712345	Computer Science	Dec 2, 2025	

xvi. TEACHER DASHBOARD

The screenshot shows the 'Teacher Dashboard' in the ProjectHub interface. The sidebar on the left has links for Dashboard (selected), Projects, Student Directory, Resources, and Settings. The main area features a welcome message 'Welcome, Anoop!' and a search bar for guided projects. There are three main sections: 'My Guided Projects' with three project cards (election voting, MCA project 2, and a new project by akash for anoop sir), 'Project Opportunities' with two cards (tdntfglglly jhygyiu and sample project hello from joyal), and a 'Calendar' for December 2025. At the bottom left, the user 'Anoop S' is logged in as a teacher.

My Guided Projects

- election voting (Owner: Akash Manikandan)
- MCA project 2 (Owner: Naveen N)
- New project by akash for anoop sir (Owner: Akash Manikandan)

Project Opportunities

- tdntfglglly jhygyiu (Owner: Naveen N)
- sample project hello from joyal (Owner: Naveen N)

Calendar

December 2025

Sun	Mon	Tue	Wed	Thu	Fri	Sat
	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30	31			

xvii. TEACHER PROJECT DASHBOARD

ProjectHub

- Dashboard
- Projects**
- Student Directory
- Resources
- Settings

Projects Dashboard

Search department projects...

Unassigned Projects (Your Department)

- tdhtfglgy jhygiyu**
Owner: Naveen N [Request to Guide](#)
- sample project hello from joyal**
Owner: Naveen N [Request to Guide](#)

All Student Projects

Project Title	Student(s)	Progress	Status	Actions
election voting	Akash Manikandan		in_progress	View Details
MCA project 2	Naveen N		in_progress	View Details
New project by akash for anoop sir	Akash Manikandan		in_progress	View Details

Anoop S
Teacher
[Logout](#)

127.0.0.1:8000/teacher_dashboard.html#

xviii. TEACHER STUDENT DIRECTORY

ProjectHub

- Dashboard
- Projects
- Student Directory**
- Resources
- Settings

Student Directory

Search students...

Students in Your Department (6)

Sort by: Name

- NN**
Naveen N
naveen@hub.com
- AM**
Akash Manikandan
akash@hub.com
- NK**
Nandhu Krishna
nandhu@hub.com
- AU**
Abhishek T U
abhishek@hub.com
- AS**
Anjali S
anjali@hub.com
- SK**
Sakshi K
sakshi@hub.com

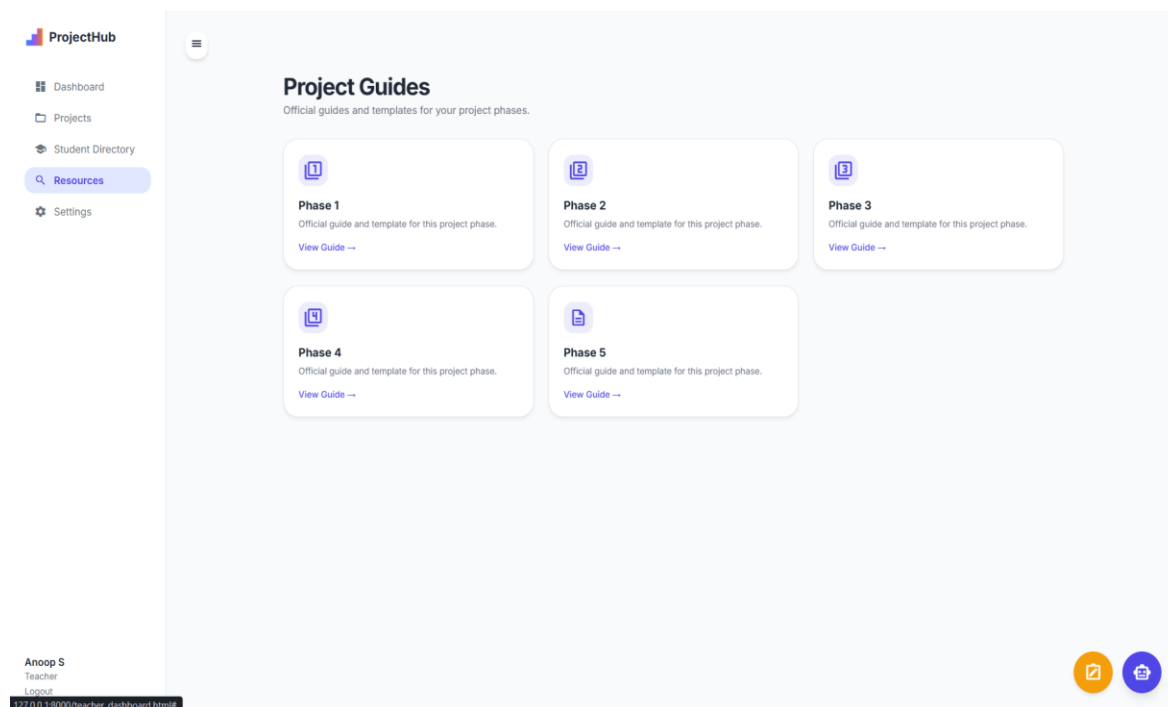
Guided Teams

- 501 election voting**
Progress:
[View Project](#)
- MCA project 2**
Progress:
[View Project](#)
- sdvsv New project by akash for anoop sir**
Progress:
[View Project](#)

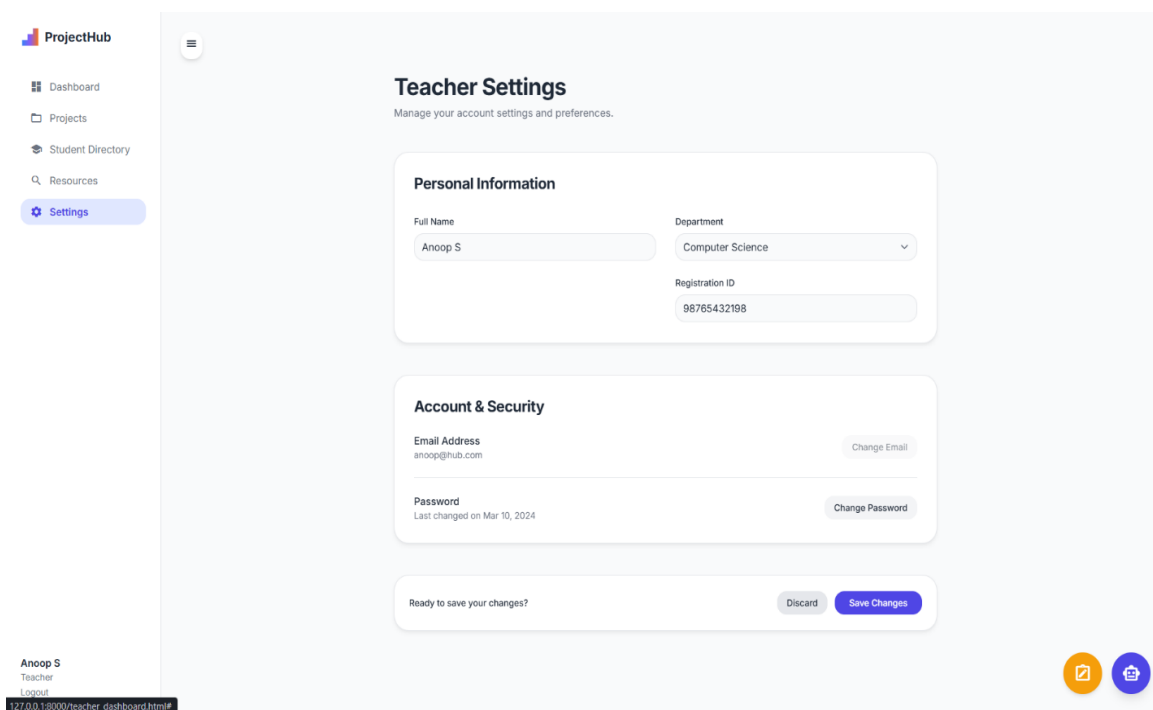
Anoop S
Teacher
[Logout](#)

127.0.0.1:8000/teacher_dashboard.html#

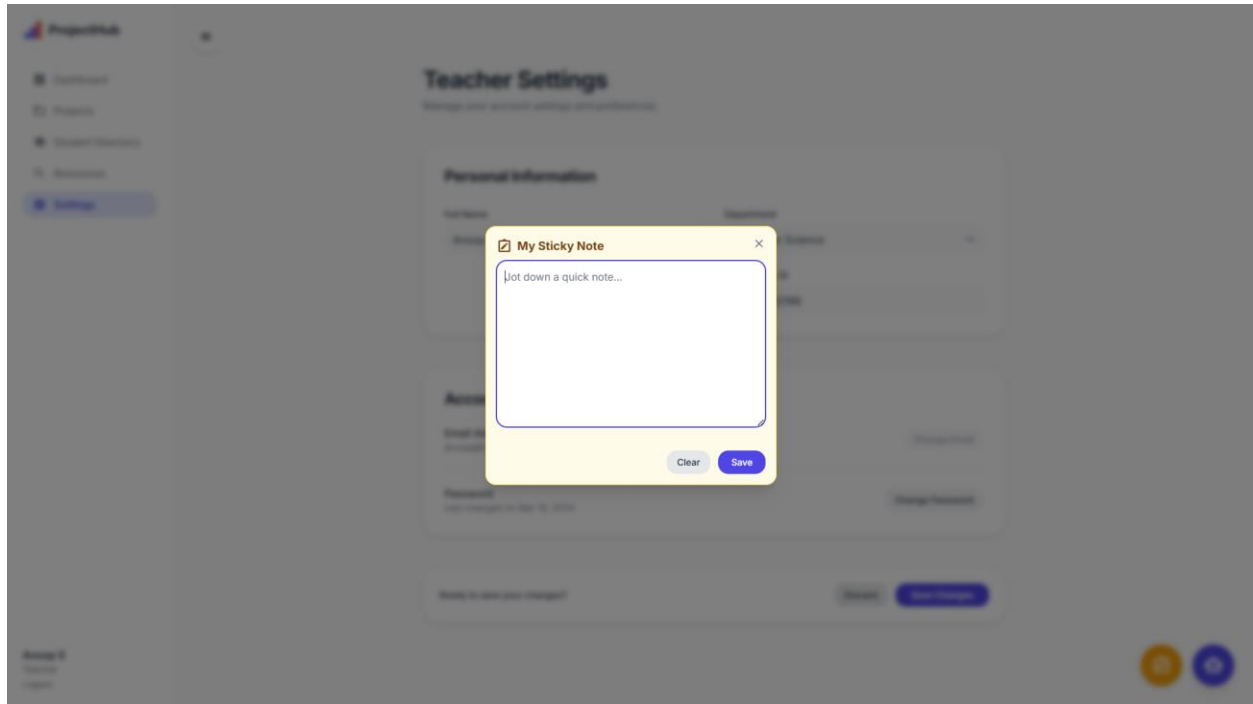
xix. TEACHER PRODUCT GUIDES



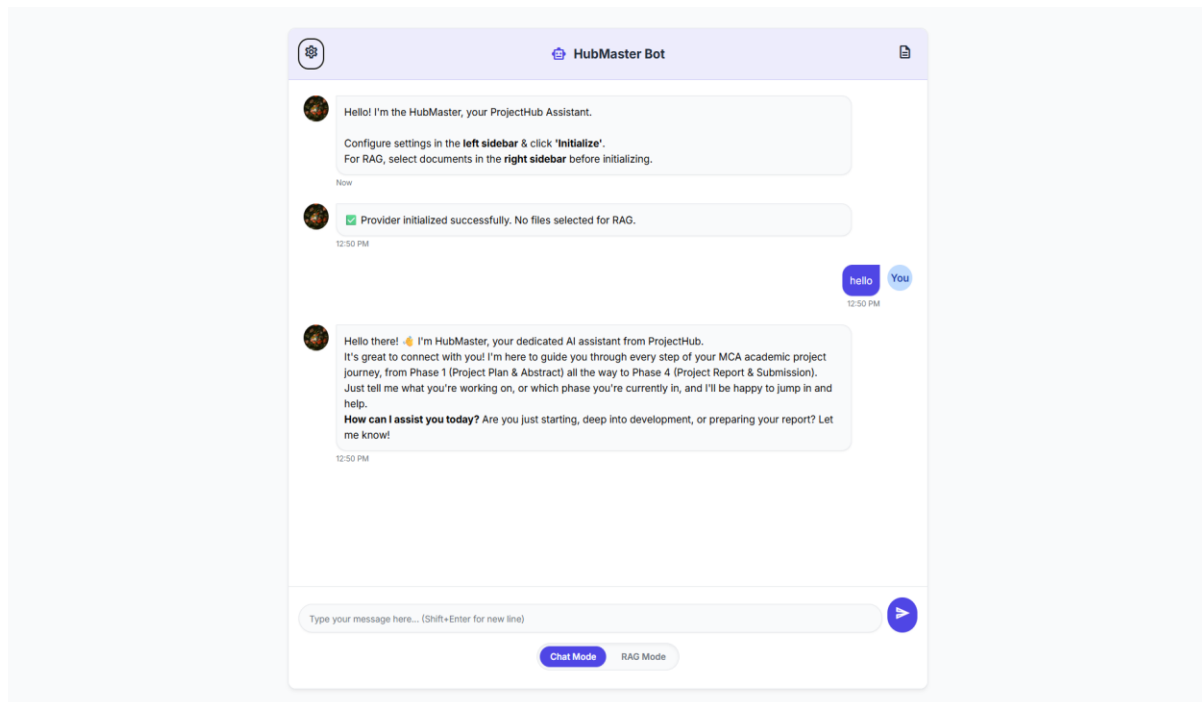
xx. TEACHER SETTINGS



xxi. TEACHER STICKY NOTES



xxii. CHATBOT



CONCLUSION

8.1 CONCLUSION

ProjectHub successfully simplifies project development in an institution across all phases by providing an interface for students to collaborate, giving monitoring access to the assigned project guides, and featuring an AI agent to give walkthroughs as well as assistance in documentation. It reduces manual workload, minimizes errors, and improves communication among students and teachers. With its user-friendly interface, secure data handling, and scalable design, the project contributes to the digital transformation of project design and development in educational institutions. Overall, this system offers a reliable and convenient platform that simplifies daily operations and creates a more organized and accessible project repository for students and teachers.

8.2 FUTURE SCOPE

The future development of the Project Hub aims to evolve the current platform into a fully integrated educational ecosystem. Key enhancements include expanding the AI capabilities to offer automated code reviews and plagiarism detection by leveraging the existing archive of "senior's projects" stored in the Vector Database. The system also plans to integrate with external platforms like GitHub for version control and Learning Management Systems (LMS) to synchronize student data and grades. To further improve accessibility and collaboration, the roadmap includes developing a native mobile application for real-time notifications and integrating video conferencing tools directly into the chat interface. Finally, the infrastructure will be scaled by migrating from Local File Storage to cloud-based solutions and implementing Multi-Factor Authentication (MFA) to ensure robust security and data availability.

REFERENCES

9.0 REFERENCES

FastAPI Documentation. (n.d.). FastAPI Features and User Guide.

Retrieved from <https://fastapi.tiangolo.com/>

MongoDB Manual. (n.d.). Introduction to MongoDB and Document Databases.

Retrieved from <https://www.mongodb.com/docs/manual/>

ChromaDB. (n.d.). Chroma: The AI-native open-source embedding database.

Retrieved from <https://docs.trychroma.com/>

Ollama. (n.d.). Get up and running with large language models locally.

Retrieved from <https://github.com/ollama/ollama>

Google AI for Developers. (n.d.). Gemini API Overview.

Retrieved from <https://ai.google.dev/docs>

MDN Web Docs. (n.d.). HTML & JavaScript: Web Development Standards. Mozilla Developer Network.

Retrieved from <https://developer.mozilla.org/>

Concepts & Architectures Lewis, P., et al. (2020).

Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. (The foundational concept for your RAG Architecture).

Postman API Platform. (n.d.). What is an API? Understanding HTTP Requests. (Relevant to your HTTP Request flow).